

eZrecon

Passive and Active Recon for the Mainframe Hunter

User Manual

Version 2.18.0

Kev Milne

<https://offensivesec.org>

Contents

Part I • Orientation	3
What eZrecon is	3
Passive and active.....	3
Keyless and keyed	4
Authorisation and scope.....	4
Part II • Installation	5
Part III • Getting started	7
The interface at a glance.....	7
Appearance and themes.....	7
Options and settings	8
Keys and settings	11
The entity graph.....	12
Help and labs.....	13
Part IV • The modules.....	15
Part V • MyMap, the nmap script runner.....	26
Part VI • The mainframe NSE scripts	30
Part VII • Methodology and playbook	39
Part VIII • Reference.....	42

Part I · Orientation

“We cannot test what we have not found.”

What eZrecon is

eZrecon is our passive-recon and OSINT tool for the mainframe hunter. It has one job: mapping the public footprint of an organisation that runs a mainframe, before we ever touch the mainframe itself. It gathers what the internet already knows and turns a single domain into a picture of the estate, the people and the exposures around it. When we do need to go active, MyMap gives us a controlled nmap runner without leaving the tool.

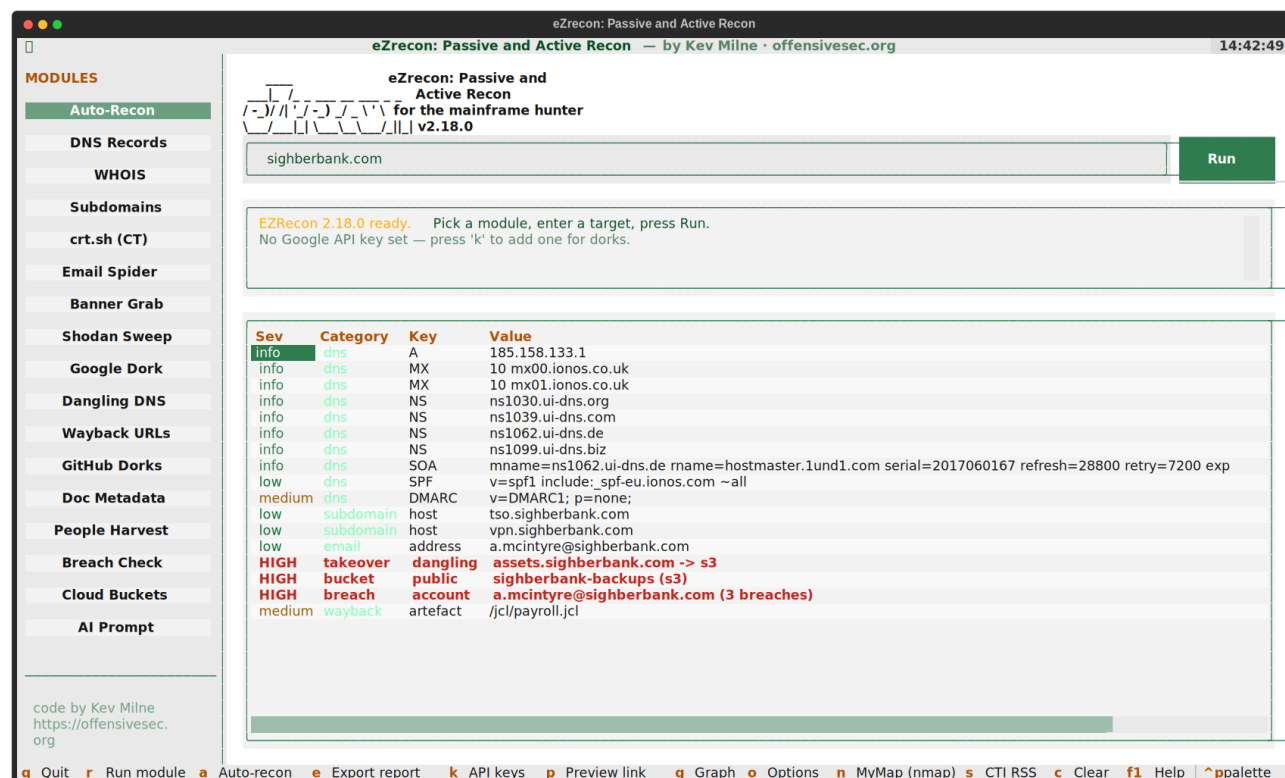


Figure 1. The main interface in the Light theme, with live DNS findings for the target and a few higher-severity leads. The module list is on the left, the target and Run at the top, and the findings table below, with high-severity rows in red.

It runs two ways. There is a full-screen terminal interface, launched with ezrecon on its own, where we pick a module, enter a target and watch findings fill a table. And there is a command line, where every capability is a subcommand we can script. This manual covers both, and everything in it works the same whichever we use, because they drive the same engine.

Passive and active

Passive recon gathers information without touching the target's own systems. Reading a certificate log, querying the web archive, searching GitHub: the target sees nothing, because we never contact them. Active recon does touch the target: a port scan connects to their host, a bucket probe queries their cloud, a MyMap scan sends packets. Active work is louder and carries more risk, both of detection and of doing harm.

eZrecon defaults to passive. The active features exist, because we often need them, but in the interface they run only when we switch active recon on, deliberately, for a target we are authorised to test. The Auto

sweep never fires an active probe on its own. Where this manual calls a module active, treat that as a note to check your scope first.

Keyless and keyed

Most of eZrecon works with no API keys at all: DNS, certificate transparency, the web archive, GitHub dork links, the PGP keyserver, favicon hashing. All keyless, all free, all quiet. A few features lean on a paid or registered service and need a key: Shodan, Have I Been Pwned, the Google search API, the GitHub code-search API. Where a feature needs a key it says so, and degrades gracefully without one, usually by handing us the query to run ourselves.

Authorisation and scope

Everything in this manual assumes we have permission to test the target. Passive recon of public sources sits on firmer ground than active probing, but the moment we act on what we find, we need authorisation, and the rules depend on where we and the target are. Keep the scope in writing, stay inside it, and when a module is active or a finding suggests an access path, stop and check that we are cleared to go further. The tool helps us find the open doors. Whether we are allowed to walk through them is on us.

NOTE The training target used throughout this manual, sighberbank.com, is a deliberately vulnerable domain. Use your own authorised targets for real work.

Safety, legality and good conduct

Two lines are worth drawing before any of this is used in anger. The first is authorisation: passive recon of public sources is one thing, but the moment we run an active module or act on a finding, we need permission in writing, and we stay inside its scope. The second is care: the active mainframe scripts drive real logon screens, and a careless brute force can lock the accounts of real people and disrupt a production system that runs a business. Go gently, prefer enumeration over brute force, keep thread counts low, and stop the moment something behaves unexpectedly. Good conduct is part of the craft, not a footnote to it.

Part II • Installation

eZrecon is a Python application. It runs anywhere Python 3.11 or later runs, which in practice means Linux, macOS, and Windows under WSL. We build and run it on Kali and on a Raspberry Pi without any fuss. This section takes us from nothing to a working install with the keys and helpers in place.

1. Check the prerequisites

We need Python 3.11 or later and pip. On a Debian-based box that is a single apt line; on macOS it comes with Homebrew's python. nmap is only needed for MyMap, the active scanner, and the tool runs fine without it until you want to scan.

```
python3 --version          # 3.11 or newer
sudo apt install python3-pip nmap #
Debian/Kali/Ubuntu
```

2. Install eZrecon

Unpack the release and install it with pip. Installing it puts the `ezrecon` command on your path and pulls in the dependencies (Textual, Rich, dnspython, aiohttp and the rest).

```
unzip EZRecon.zip -d ezrecon && cd ezrecon
pip install . --break-system-packages ezrecon --version
```

3. Launch it

Run it with no arguments for the full-screen interface, or give it a subcommand to drive a single capability from the shell. If the interface looks cramped, make the terminal window bigger; it is happiest at 120 columns or more.

```
ezrecon                  # the interface
ezrecon auto sithuberbank.com # or straight from the shell
```

4. Add your keys (optional)

Most of eZrecon is keyless. For the few features that need a key (Shodan, Have I Been Pwned, the Google search API, the GitHub code-search API) press k in the interface, or set them from the shell. They are stored once in the config vault and masked in the interface.

```
ezrecon config set shodan_api_key <key>
ezrecon config set hibp_api_key <key>
ezrecon config set github_token <token>
```

5. Stand up SearXNG for keyless dorking (optional)

The Google search API is closed to new customers, so for the dork and preview features we recommend a local SearXNG. One command builds it, enables the JSON API, and points eZrecon at it.

```
ezrecon searx setup      # build and start it
ezrecon searx status     # check it
ezrecon config set-setting searx_pace 3 # slow it if engines rate-limit
```

6. Confirm nmap for MyMap (optional)

MyMap drives your installed nmap and reads its NSE scripts from disk. If nmap is present, MyMap lists every script by category the moment you open it. If it is not, install it and reopen.

```
nmap --version ls /usr/share/nmap/scripts | wc -l # the scripts MyMap will list
```

NOTE If the interface looks cramped or wraps oddly, widen the terminal. eZrecon is happiest at 120 columns or more, and MyMap likes a little more still.

Part III · Getting started

The interface at a glance

The banner sits top left, with the version beside it. Down the left is the module list, and beneath it the Graph, Options, API keys and Help buttons, with the attribution pinned at the bottom. Across the top of the main pane is the target box and the Run button. Below that is a status log that streams what each stage is doing, and beneath that the findings table: severity, category, key and value. Everything we run in a session accumulates there. We drive it from the keyboard: a runs the Auto sweep, r the selected module, g the graph, n MyMap, and F1 help.

Appearance and themes

Press o for Options and the Appearance section is at the top. Four themes ship: Phosphor, the default green-on-black CRT look; Amber, the same idea in amber; Light, black on white for bright rooms and projectors; and Mono, high-contrast white on black. Pick one and it applies immediately and is remembered. The theme also drives how findings are coloured, so severities stay readable whichever we choose, and high findings are always drawn in red.

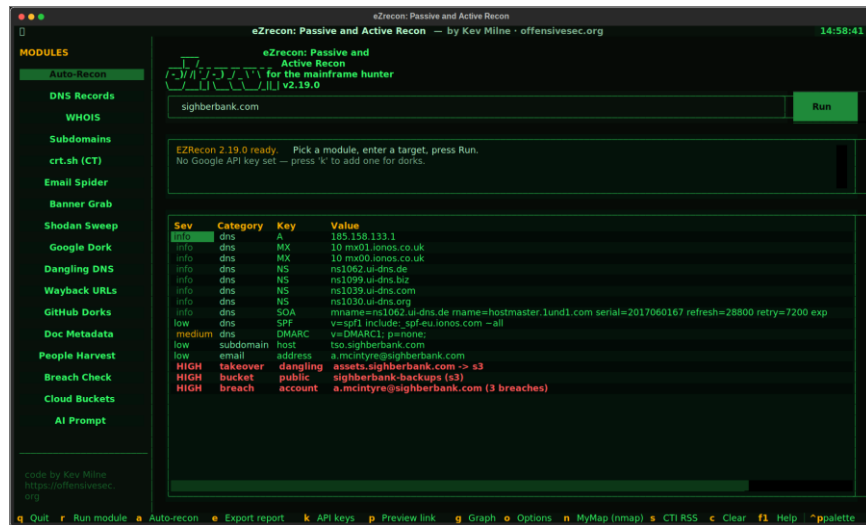


Figure 2. The default Phosphor theme — green on black, the classic terminal look.

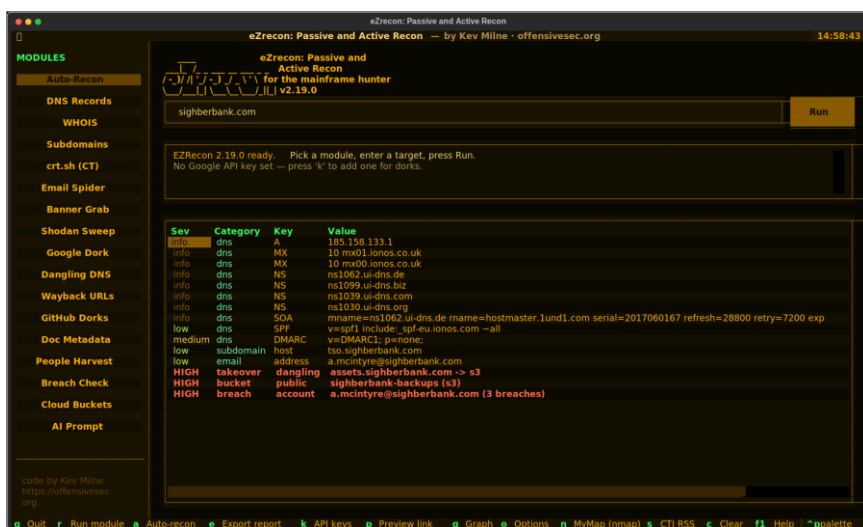


Figure 3. The Amber CRT theme, the same idea in amber for a warmer phosphor feel.

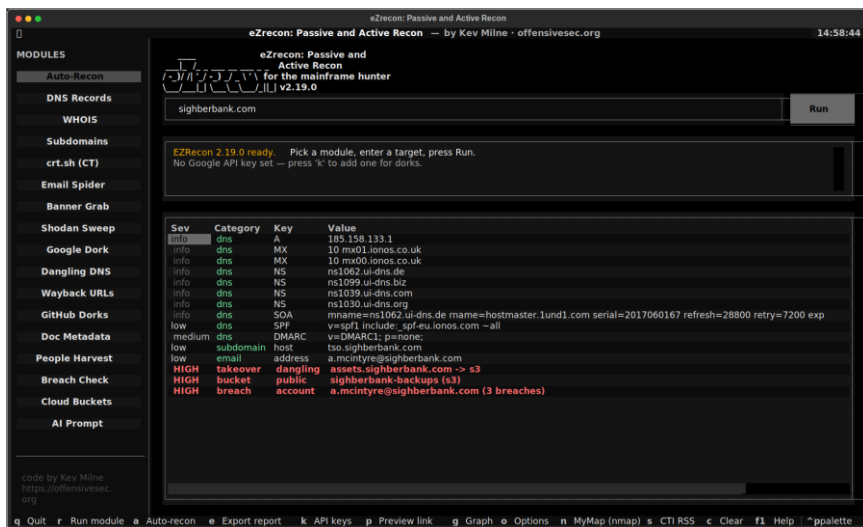


Figure 4. The Mono theme — high-contrast white on black for maximum legibility.

Options and settings

Options (press o) collects the run-time settings. Most have sensible defaults and you can ignore them until a job calls for something different; this section explains what each one does, why you would change it, and what you are trading off.

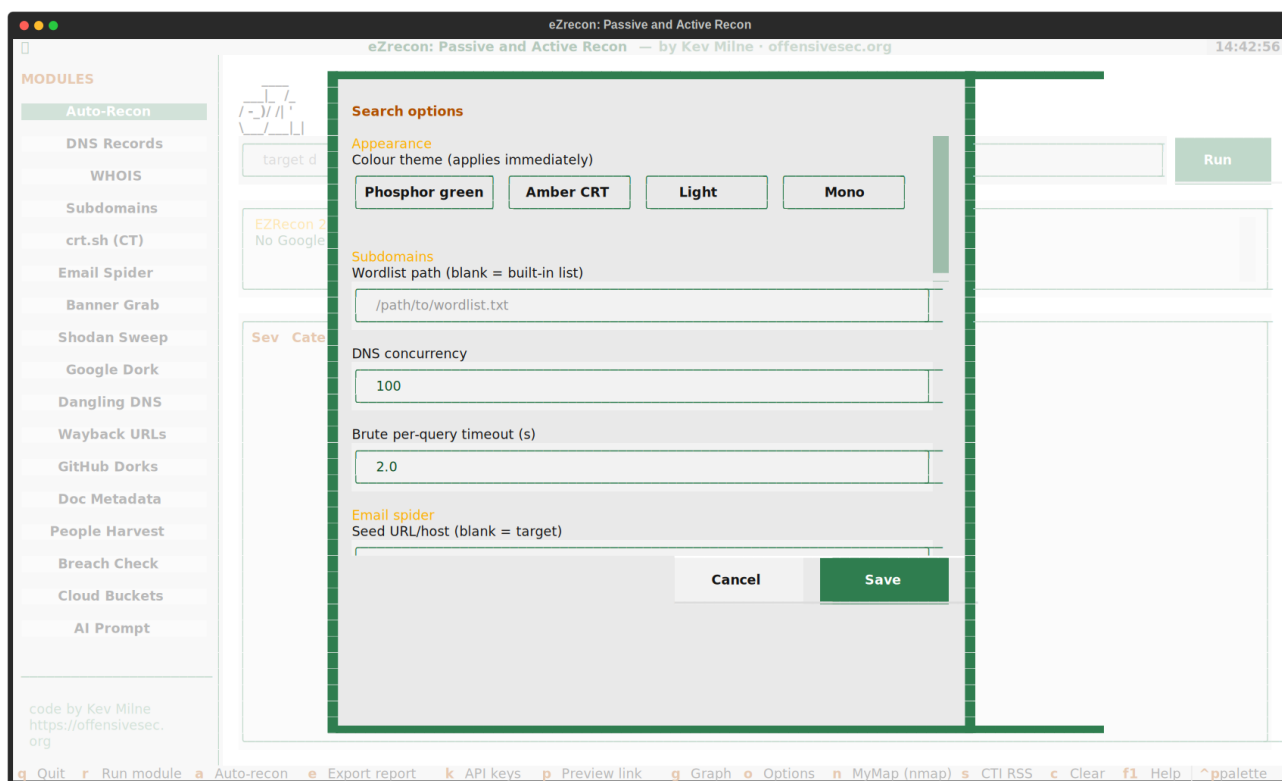


Figure 5. Options, with the four themes across the top and the run-time settings below. Behind the popup, the high-severity findings stay red in every theme.

Appearance

Colour theme — Phosphor, Amber, Light or Mono. Purely cosmetic, applied immediately and remembered. Use Light for a projector or a bright room, Mono for maximum contrast, and the phosphor greens for everything else. Severity colours stay readable in all four.

Subdomains

Wordlist path — The list the subdomain brute force works through. Blank uses the built-in list, which is tuned with mainframe names. Point it at a bigger list when you want depth and have the time; a larger list finds more but takes longer and makes more DNS traffic.

DNS concurrency — How many DNS lookups run at once. Higher is faster but heavier on your resolver and easier to notice. The default is a good balance; drop it on a flaky link, raise it when you are resolving a very large list and the resolver can take it.

Brute per-query timeout — How long to wait for a single DNS answer before giving up on it. Raise it on a slow or distant resolver so you do not miss hosts that were simply slow to answer; lower it to move faster when the resolver is quick.

Email spider

Seed URL or host — Where the spider starts. Blank starts at the target. Point it at a specific subdomain (a careers site, a support portal) when that is where the addresses live.

Crawl depth — How many links deep the spider follows from the seed. Deeper finds more but wanders further from the start and takes longer. Two or three is usually plenty for a corporate site.

Max pages — A hard cap on pages fetched, so the spider cannot run away on a large site. Raise it for a thorough crawl, lower it to keep things quick.

Spider concurrency — How many pages are fetched at once. Higher is faster but hits the target's web server harder; keep it modest to stay polite and quiet.

Known emails — Addresses you already have, added straight into the dossier. These seed the naming-convention inference, so even a couple of confirmed addresses sharpen the generated ones.

Ports (banner grab)

Ports — The ports the banner grab connects to. Blank uses the mainframe profile: TN3270 on 23 and 992, DRDA and DB2 on 446 and 50000, and the common web and admin ports. Set your own list to focus on, or add to, that profile. Remember this is active.

Port concurrency — How many ports are probed at once. Higher is faster and noisier; a modest number is less likely to trip a threshold on the target.

Preview

Engine — Which search backend the preview and dork features use: auto picks what is available, ddg uses DuckDuckGo, api uses your configured key or SearXNG. Set it explicitly when you want to force one path.

Result limit — How many results a preview returns. Raise it to cast wider, lower it to keep the list scannable.

Chaining

Auto-chain discovered hosts — When on, a discovered subdomain can feed the next stage automatically, so one run flows into the next. Turn it off when you want tight control over exactly what runs against what.

Chain cap — An upper bound on how many subdomains chaining will pursue, so a domain with hundreds of hosts cannot set off an enormous run. Raise it for a thorough sweep, lower it to stay contained.

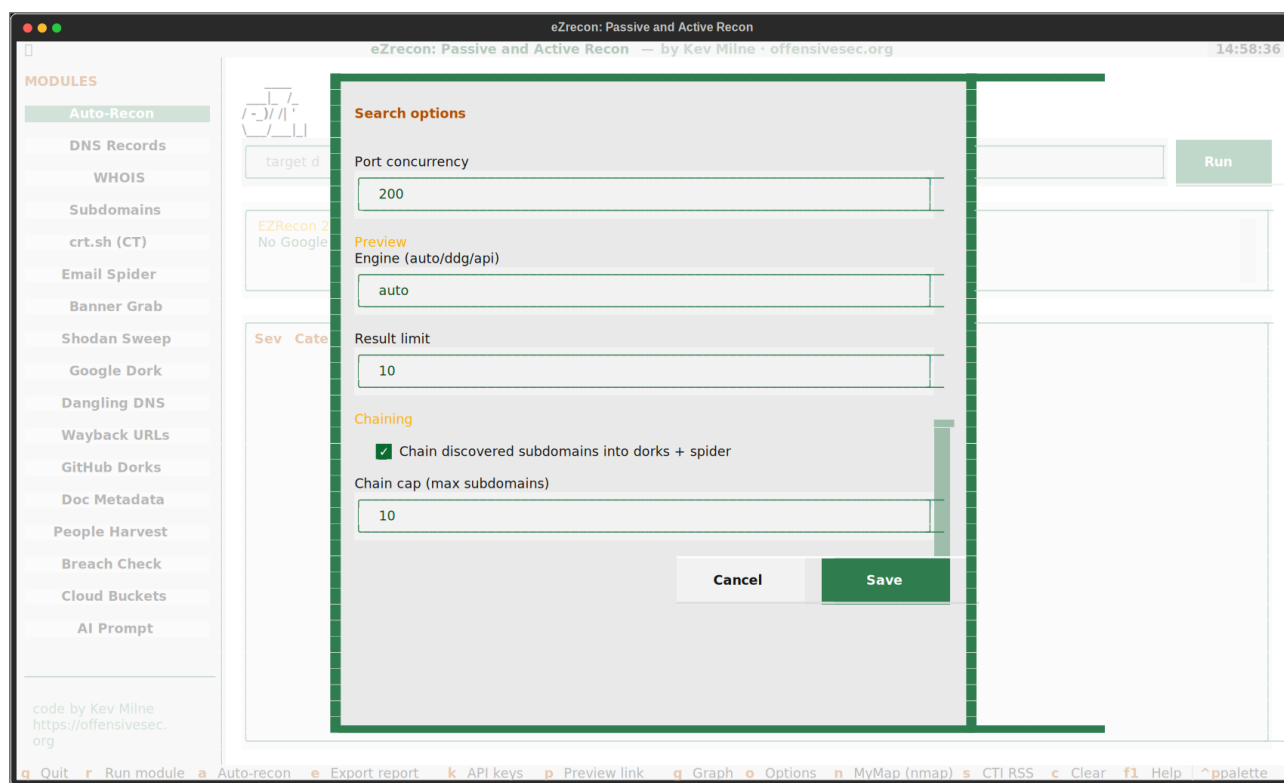


Figure 6. The lower half of Options: the email spider, the banner-grab port profile, the preview engine, and the chaining controls.

Keys and settings

Press **k** for the API keys screen to enter the Google, Shodan, HIBP and GitHub keys. Keys are secret and masked; they live in the config vault, so we set them once and never paste them into a command. From the shell the same vault is config set for secrets and config set-setting for settings.

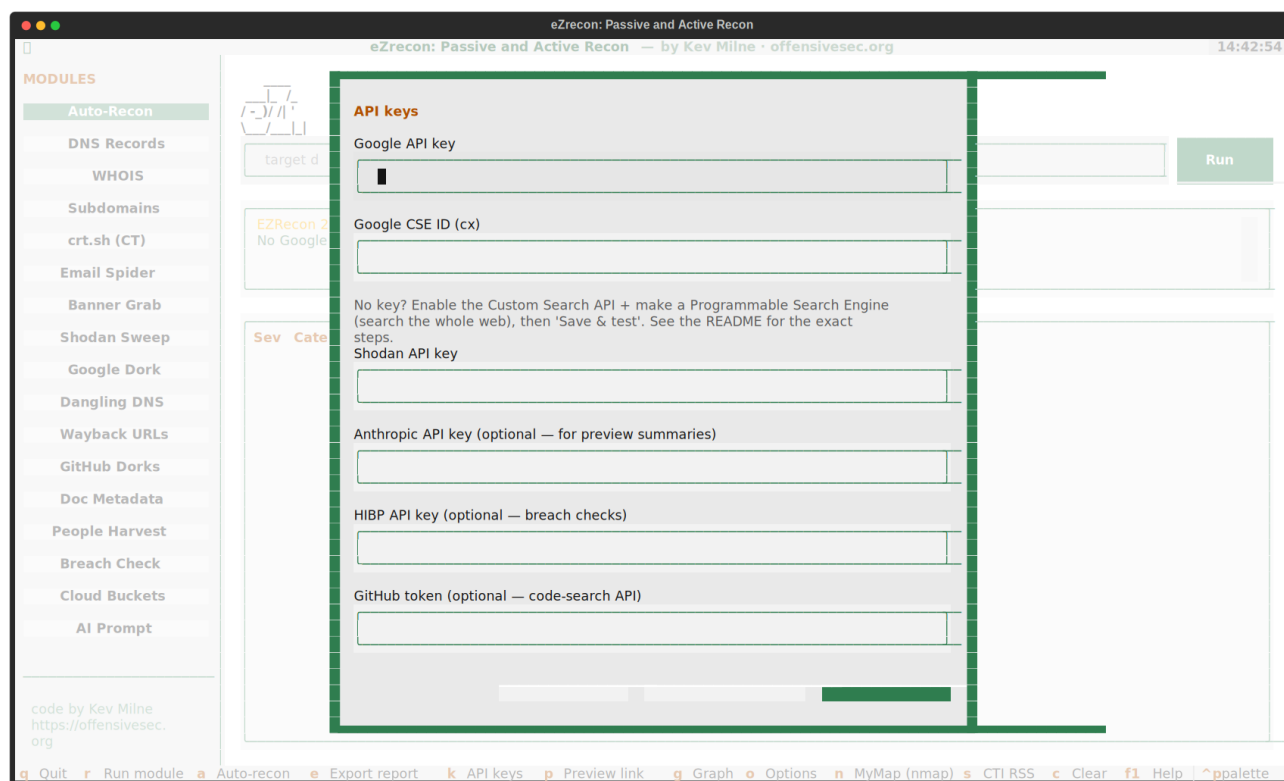


Figure 7. The API-key vault. Keys are entered once, masked, and stored in the config; nothing is ever pasted into a command.

Findings, severity and the session

Every module writes findings into the current session. A finding has a category, a key, a value, a source and a severity. Severity is how eZrecon tells us what matters, and when we are triaging a busy session we read the reds first. The levels are defined in the reference.

The entity graph

Press **g** for the graph. Everything in the session becomes a node (domains, subdomains, IPs, emails, people, documents, buckets, breaches) coloured by type and joined by typed edges. This is where recon stops being a list and becomes a map. From most nodes we can pivot, running the next logical step straight from the node.

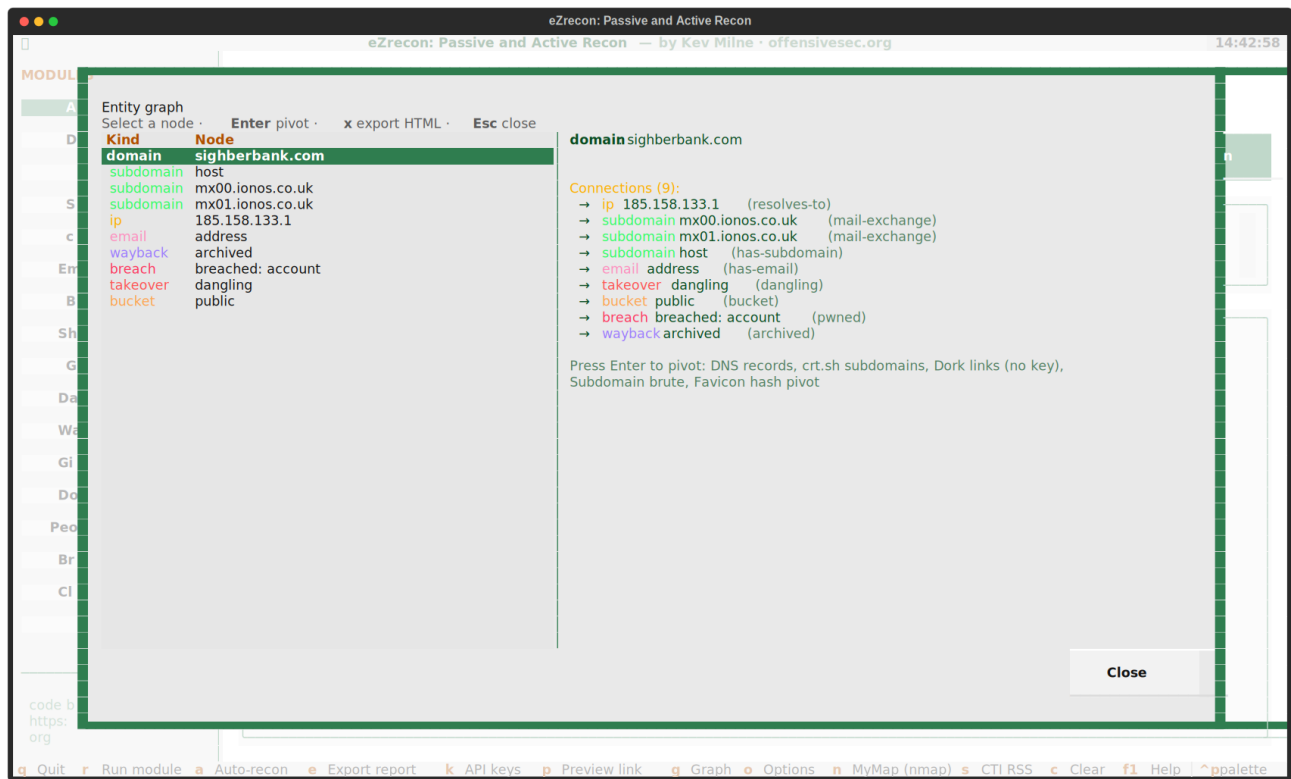


Figure 8. The entity graph. The target is selected on the left and its nine connections are on the right, each a typed edge to an IP, a mail exchanger, a person, a breach, a takeover candidate or an archived artefact.

Help and labs

eZrecon carries its own help. The Help / Labs item on the left, and the F1 key anywhere, open a help screen with a topic for every feature. Each topic explains what the feature is, how to use it, and finishes with a short hands-on lab you can run against a domain you own.

F1 is contextual. Press it on the main screen and it opens the help for the module you have selected. Press it inside MyMap while your cursor is on the command line and it points you at the fact that the line is editable; press it on the script list and it explains how selection works. The idea is that help speaks to wherever you actually are.

The labs are the fastest way to learn the tool. Start with Lab 0 on the overview topic: type `sighberbank.com`, press `a` for a full passive sweep, then `g` to see the findings as a graph. Each module's lab builds a habit worth keeping, from reading the reds first to pivoting from a document author to an address.

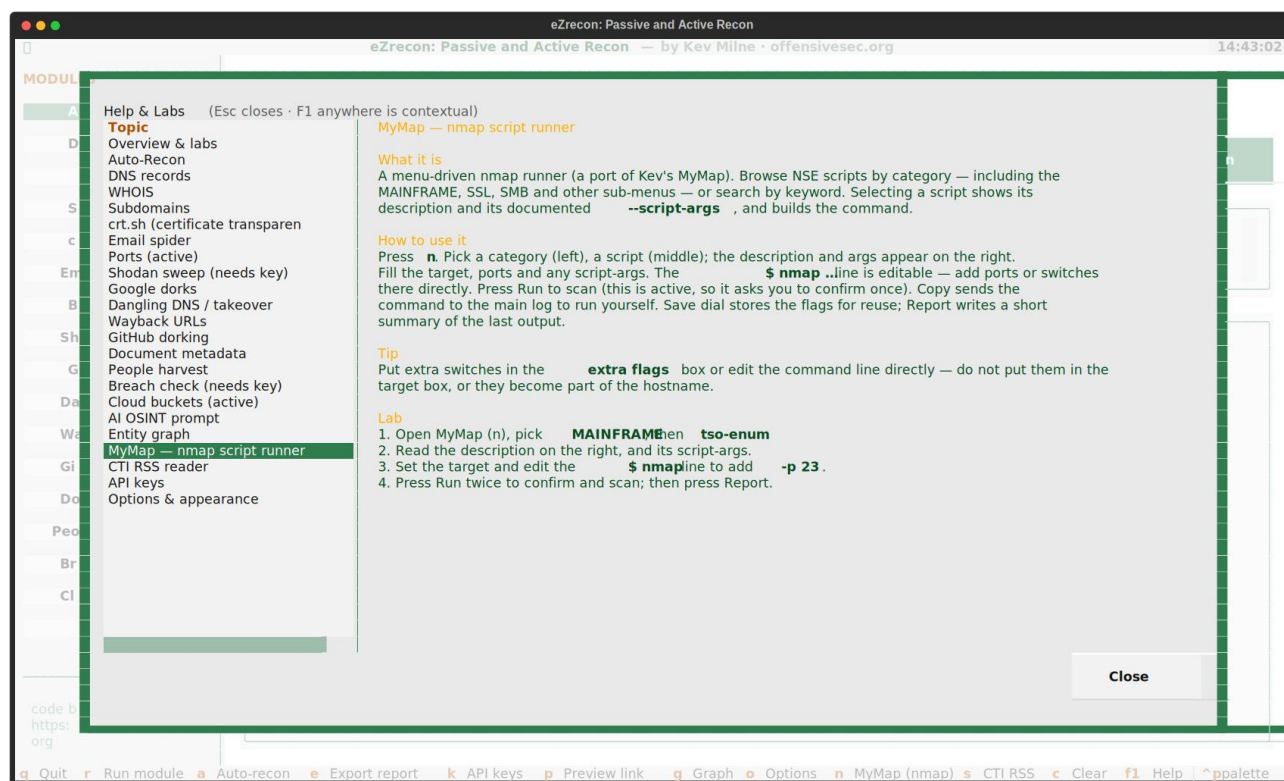


Figure 9. The in-app help. Every feature has a topic with what it is, how to use it, and a short lab. F1 opens this contextually to wherever your cursor is.

Reporting and export

Press **e** to export the current session. eZrecon writes the findings and the built queries out in the formats you would want for a report: Markdown and HTML for prose, JSON for tooling, and a queries file you can re-run. The entity graph can be exported to a standalone HTML file from the graph screen, which is a good figure for a report because it shows how the findings connect rather than just listing them. Everything you export is drawn from the same session, so the evidence trail behind each claim stays intact.

Part IV · The modules

Each module answers the same three questions: what it does, how we run it, and why we would. They are grouped by the job they do, in roughly the order we use them on an engagement.

Foundation — mapping the estate

These are the first things we run. They turn a bare domain into a picture of the organisation's public footprint: its DNS, who registered it, the hosts hanging off it, and the network it lives on. None of them touch the target in a way that looks like an attack, and none need a key.

DNS records

Resolves the standard record types (A, AAAA, MX, NS, TXT, SOA, CNAME) and tries a zone transfer against each nameserver.

Why we run it. DNS is the skeleton of the estate. MX shows us the mail provider, TXT leaks the SPF and DKIM entries that name third parties, and a nameserver that allows a zone transfer hands us the whole internal map in one request. On a mainframe job the TXT and CNAME entries often point at the middleware and gateways sitting in front of z/OS.

What you're looking for. The mail provider in MX, third-party services named in TXT and SPF, and any nameserver that answers a zone transfer.

Run it

```
ezrecon dns <domain>
```

In the interface: DNS Records module, then Run

Zone transfers almost always fail, but the one in fifty that works is worth the check every time.

WHOIS

Pulls the domain registration record: registrar, key dates, and the registrant organisation and contacts where they are not redacted.

Why we run it. Registration data ties a domain to an organisation and to other domains the same entity owns. Even where privacy protection hides the contact, the registrar, the creation date and the nameservers help us confirm ownership before we go any further.

What you're looking for. Confirmation of ownership, the registrar and dates, and other domains the same organisation registered.

Run it

```
ezrecon whois <domain>
```

In the interface: WHOIS module

Subdomains

Brute-forces subdomains from a wordlist tuned with mainframe names, resolving each and keeping the ones that answer.

Why we run it. Organisations expose far more than their main site: vpn, mail, dev, test, mainframe, tso, and the rest. Each resolved host is a new target and a new pivot, and the wordlist carries mainframe-relevant names as well as the usual web estate.

What you're looking for. Live hosts that widen the estate — gateways, mail, dev and test, and anything with a mainframe-looking name.

Run it

```
ezrecon subdomains <domain>
```

In the interface: Subdomains module

Point it at a bigger wordlist in Options when you want depth over speed.

crt.sh (certificate transparency)

Reads certificate transparency logs for every hostname ever issued a certificate under the domain.

Why we run it. Every publicly trusted certificate is logged, which makes CT the single richest passive source of subdomains, including internal-looking names we would never guess by brute force. It costs the target nothing because we are reading a public log, not probing their servers.

What you're looking for. Internal-looking hostnames you would never guess: names that appear only because a certificate was issued.

Run it

```
ezrecon crtsh <domain>
```

In the interface: crt.sh (CT) module

ASN enrichment

Maps IP addresses to their autonomous system and netblock using Team Cymru's DNS service.

Why we run it. Knowing the ASN tells us who hosts the target and how big their address space is. A block the organisation owns is very different from one owned by a cloud provider, and the netblock gives us a range to sweep for more of their estate.

What you're looking for. Whether the organisation owns its address space, and the netblock that gives you a range to explore.

Run it

```
ezrecon asn <domain-or-ip>
```

In the interface: runs inside Auto, or as a pivot on IP nodes in the graph

Service intelligence — what is listening

Once we know the hosts, these tell us what runs on them and where else those services live. The port scan is active. Shodan and the favicon lookup lean on third parties so we stay quiet.

Ports

Connects to a mainframe-relevant port profile and grabs banners, defaulting to TN3270 on 23 and 992, DRDA and DB2 on 446 and 50000, and the common web and admin ports.

Why we run it. A banner confirms what a host actually is. Finding TN3270 listening tells us there is a terminal front end to reach; DRDA tells us there is a DB2 subsystem. This is where passive mapping turns into a concrete attack surface.

What you're looking for. Which mainframe services actually answer — TN3270 on 23 and 992, DRDA or DB2 on 446 and 50000 — and what their banners say.

Run it

```
ezrecon ports <host>
```

In the interface: Ports module (active)

This one connects to the target, so it is active. In the interface it runs only once you enable active recon.

Shodan sweep

Runs a mainframe-flavoured Shodan search and returns matching hosts, services and banners from Shodan's existing scan data.

Because Shodan has already scanned the internet, this is reconnaissance without a packet. The queries are tuned for the fingerprints that matter here — TN3270, DRDA, midrange and mainframe banners — so a sweep can confirm an exposed terminal service before you ever connect.

Why we run it. Shodan has already scanned the internet, so we learn what is exposed without sending a single packet to the target. The queries are tuned for mainframe and midrange fingerprints.

What you're looking for. Exposed hosts and mainframe or midrange fingerprints Shodan already holds, without you sending a packet.

Run it

```
ezrecon shodan <domain>
```

In the interface: Shodan module (needs a key)

Add the key with the keys screen, or `ezrecon config set shodan\_api\_key`.

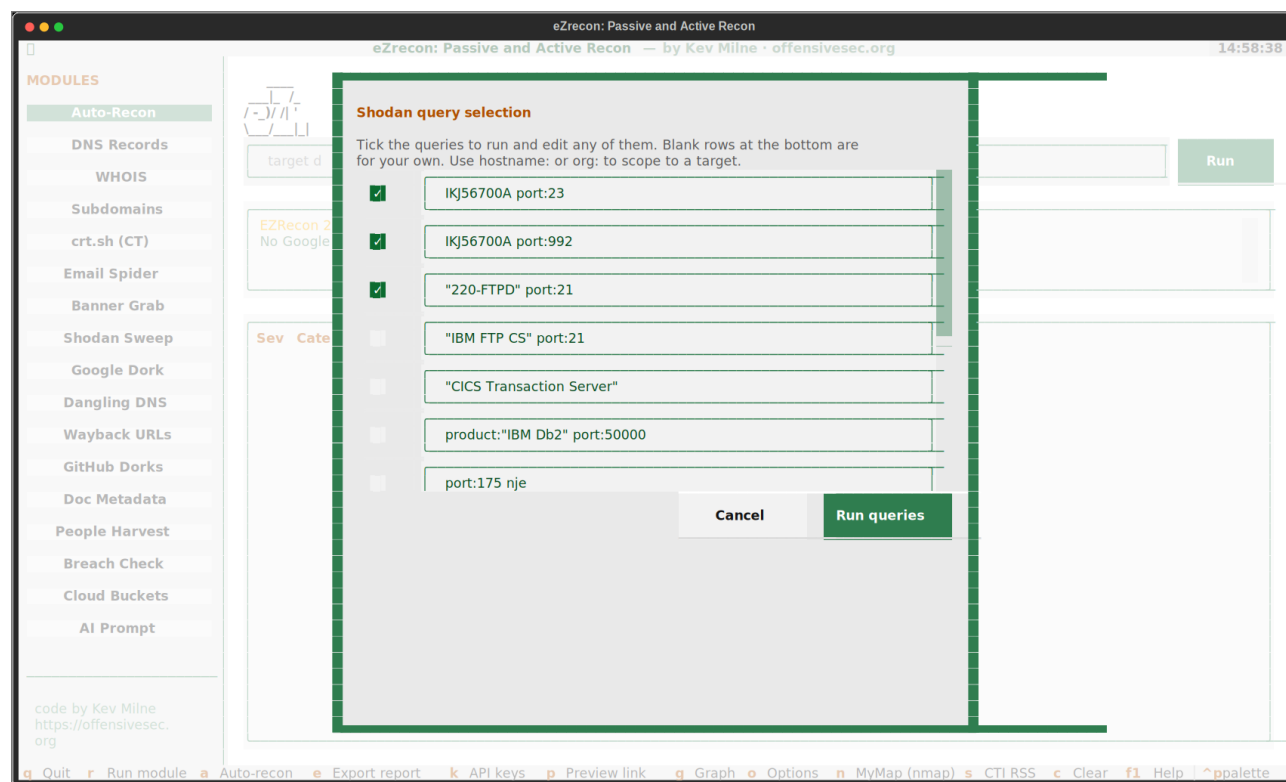


Figure 10. The Shodan sweep. eZrecon builds mainframe-flavoured queries; with a key it returns the hosts and banners Shodan already holds, with no packet sent to the target.

Favicon hash pivot

Hashes the site's favicon the way Shodan does and builds the http.favicon.hash query. With a Shodan key it pivots straight to every other host serving the same icon.

Why we run it. A shared favicon links hosts that otherwise look unrelated: the real server behind a CDN, staging and development clones, and phishing copies that reuse a brand's icon. It is one of the cheapest ways to turn one asset into a whole estate.

What you're looking for. Other hosts serving the same icon — the real server behind a CDN, staging clones, and brand-reuse.

Run it

```
ezrecon favicon <host> [--shodan]
```

In the interface: Favicon pivot on domain, subdomain and IP nodes in the graph

The hash maths is self-contained, so the query works without a key. The pivot to live hosts needs Shodan.

Search and dorking — what the web already knows

These use search engines and code hosts to find things that are indexed and public but that nobody meant to advertise. Dorking is the art of asking a precise question so the noise falls away.

Google dorks

Builds and, with a key or SearXNG, runs a catalogue of mainframe-focused dorks: exposed listings, config and log files, error pages that leak internals, and mainframe filetypes.

The catalogue is not generic. It carries queries aimed at the things a mainframe shop leaks: JCL and COBOL filetypes, exposed listings and log files, and error pages that spill hostnames. You tick the ones you want, edit any of them, and either open them as links or, with a key or SearXNG, run them and record the hits.

Why we run it. Search engines index things developers forget are public. A well-aimed dork surfaces a directory listing, a stack trace with a hostname in it, or a JCL file sitting on a web root. Without a key you still get the clickable queries to run by hand.

What you're looking for. Directory listings, config and log files, error pages that leak internals, and JCL or COBOL on a web root.

Run it

```
ezrecon dork <domain>
```

In the interface: Google Dork module

The Google Programmable Search API is closed to new customers, so we recommend the keyless SearXNG path. Stand one up with `ezrecon searx setup`.

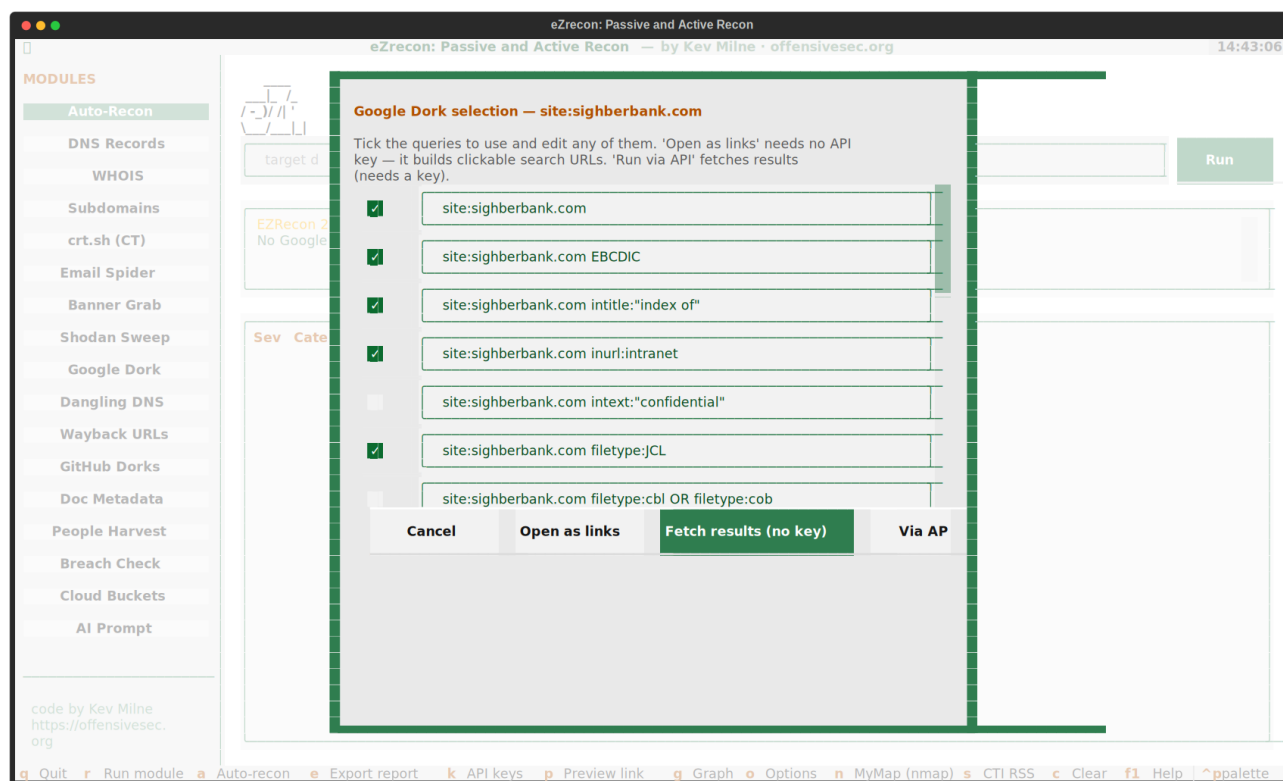


Figure 11. The Google Dork selection, pre-loaded with mainframe-focused queries. Tick the ones you want, edit any of them, and open them as links or run them through a key or SearXNG.

SearXNG

Bootstraps a local SearXNG instance in one command, with the JSON API enabled, and points eZrecon at it.

Why we run it. A self-hosted SearXNG gives us a reliable, private search backend that the dork and preview features lean on without a key. It handles the container and the config for us across Linux, WSL and macOS.

Run it

```
ezrecon searx setup | status | stop
```

In the interface: configured once, then used by the dork and preview features

If a run comes back empty, the engines have rate-limited themselves on a burst. eZrecon paces the queries and retries; you can widen the pace with `config set-setting searx\_pace 3`.

GitHub dorking

Builds GitHub code-search queries scoped to the target (JCL, COBOL, connection strings, .env and key files) and returns them as links. With a token, --fetch runs them through the API and records the hits.

Developers commit things they should not, and a mainframe shop's things are copybooks, JCL, connection strings and credentials. Scoping the search to the organisation's name or repos cuts the noise; in link mode it is keyless, and with a token it runs the searches and records what it finds.

Why we run it. Public repositories leak exactly the things a mainframe shop least wants public: source, copybooks, connection strings, credentials. Scoping the search to the target's org or name cuts straight to the leaks.

What you're looking for. Leaked source, copybooks, connection strings and keys in public repositories scoped to the target.

Run it

```
ezrecon github <domain> [--org <org>] [--fetch]
```

In the interface: GitHub Dorks module

Link mode is keyless. The API fetch needs a token in github_token.

Preview

Runs a raw web search through the configured engine and shows the results, so you can look without committing to a whole dork run.

Why we run it. Sometimes we just want to look, not run a module. Preview is the quick lookup for a name, a string, or a hunch.

Run it

```
ezrecon preview <query>
```

In the interface: Preview screen (press p)

History and documents — the past and the paperwork

The web remembers things that are no longer live, and documents carry more than their contents. These two mine the archive and the metadata.

Wayback URLs

Pulls every URL ever archived under the domain from the Internet Archive and flags the interesting ones: config and secret files, backups, exposed version control, admin paths, and mainframe artefacts like JCL and COBOL.

The value here is time. A file that was linked in 2019 and deleted in 2020 is invisible to a scanner today, but the archive still holds the URL and often the content. eZrecon pulls the whole history and sorts it, weighting the mainframe artefacts and the secret files to the top so you are not reading ten thousand image URLs.

Why we run it. The archive holds files that were deleted or unlinked years ago but were captured while they were live. Active scanning cannot see them because they are gone; the archive still has the URL, and often the content. For a mainframe target the JCL and copybook hits are gold.

Run it

```
ezrecon wayback <domain>
```

In the interface: Wayback URLs module

Keyless, and invisible to the target, because we read a public index.

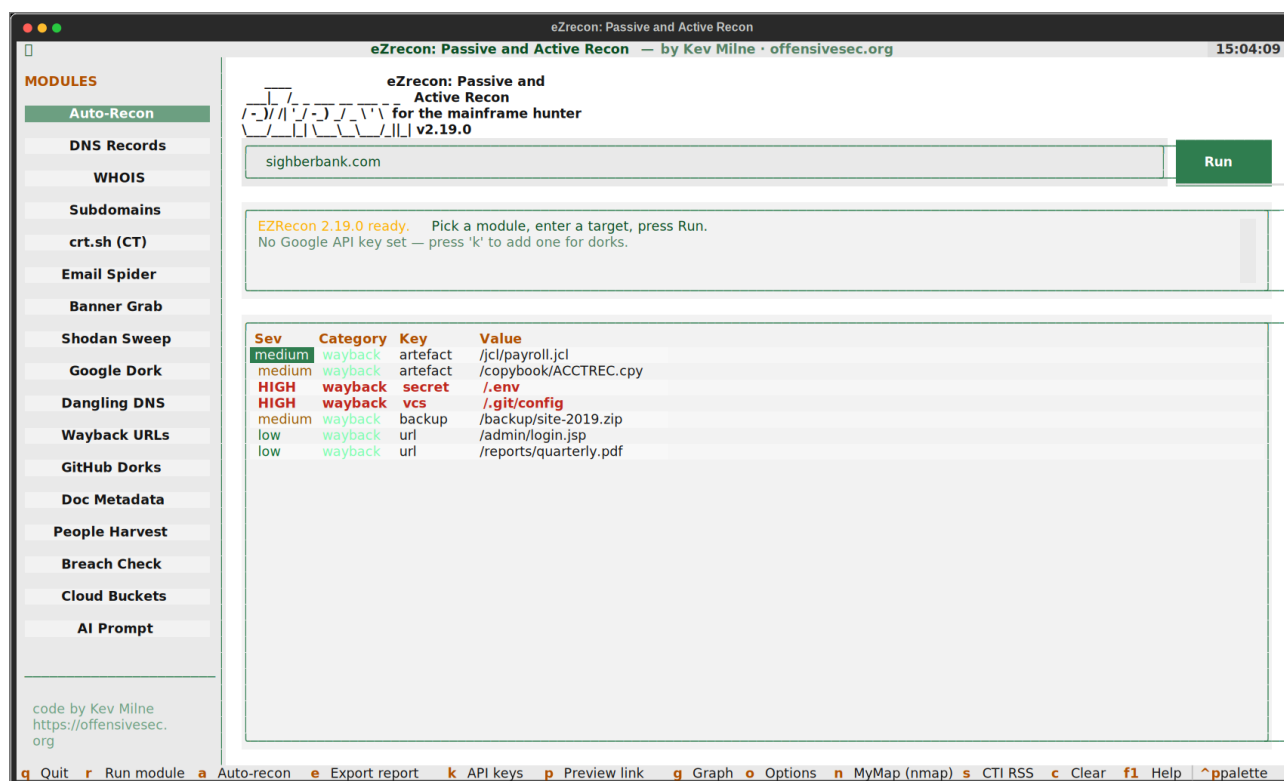


Figure 12. A Wayback harvest with the interesting paths flagged: JCL and copybook artefacts, an exposed .env and .git, a backup archive, and admin paths — several of them things active scanning could never see.

Document metadata

Fetches the public PDFs and Office documents we have found and extracts their metadata: author and last-modified-by usernames, the authoring software and version, and company fields.

Every published PDF and Office document carries more than its text. The author and last-modified-by fields are usernames, and on z/OS a username is a candidate TSO or RACF ID; the software and company fields fill in the picture. A single runbook can hand you a real userid.

Why we run it. Documents leak internal detail in their metadata. Author names give us the organisation's username convention and real people; a runbook's author becomes a userid for our TSO and RACF lists.

What you're looking for. Author and last-modified-by usernames — candidate TSO and RACF userids — and the software and company fields.

Run it

```
ezrecon docmeta <domain> | --url <doc>
```

In the interface: Doc Metadata module

It reads the documents that dork and Wayback already surfaced, so run those first, or point it straight at a URL.

People — the human attack surface

Mainframe access starts with a userid. These three find the people, work out how their addresses are formed, and check whether their credentials are already floating around from someone else's breach.

Email spider

Crawls the target's own site to a shallow depth and scrapes the email addresses it finds.

Why we run it. The organisation's own pages are the most reliable source of real addresses, and those addresses seed the naming convention and the breach checks that follow.

What you're looking for. Real addresses on the organisation's own pages, which seed the naming convention and the breach checks.

Run it

```
ezrecon email <domain>
```

In the interface: runs inside Auto and the harvest chain

People harvest

Consolidates every email and name we have gathered, dedupes them with the source that found each, adds a keyless PGP keyserver lookup, infers the organisation's email-naming convention, and generates the likely addresses for names it has but has not confirmed.

The trick is the pattern. Given a few confirmed addresses it works out how the organisation forms them — first.last, f.last, an employee number — and then it can generate the address for any name you hand it. Those addresses are candidate RACF user IDs and phishing targets, which is where an OSINT dossier becomes an access path.

Why we run it. Intel is scattered across the spider, document metadata and certificate subjects. Harvest pulls it into one dossier and, crucially, works out the pattern so that a handful of known addresses plus a few names gives us addresses for everyone. Those addresses are candidate userids and phishing targets.

What you're looking for. The naming pattern that turns names into addresses, and a consolidated, de-duplicated dossier of people and userids.

Run it

```
ezrecon harvest <domain> [--deep] [--names "First Last"]
```

In the interface: People Harvest module

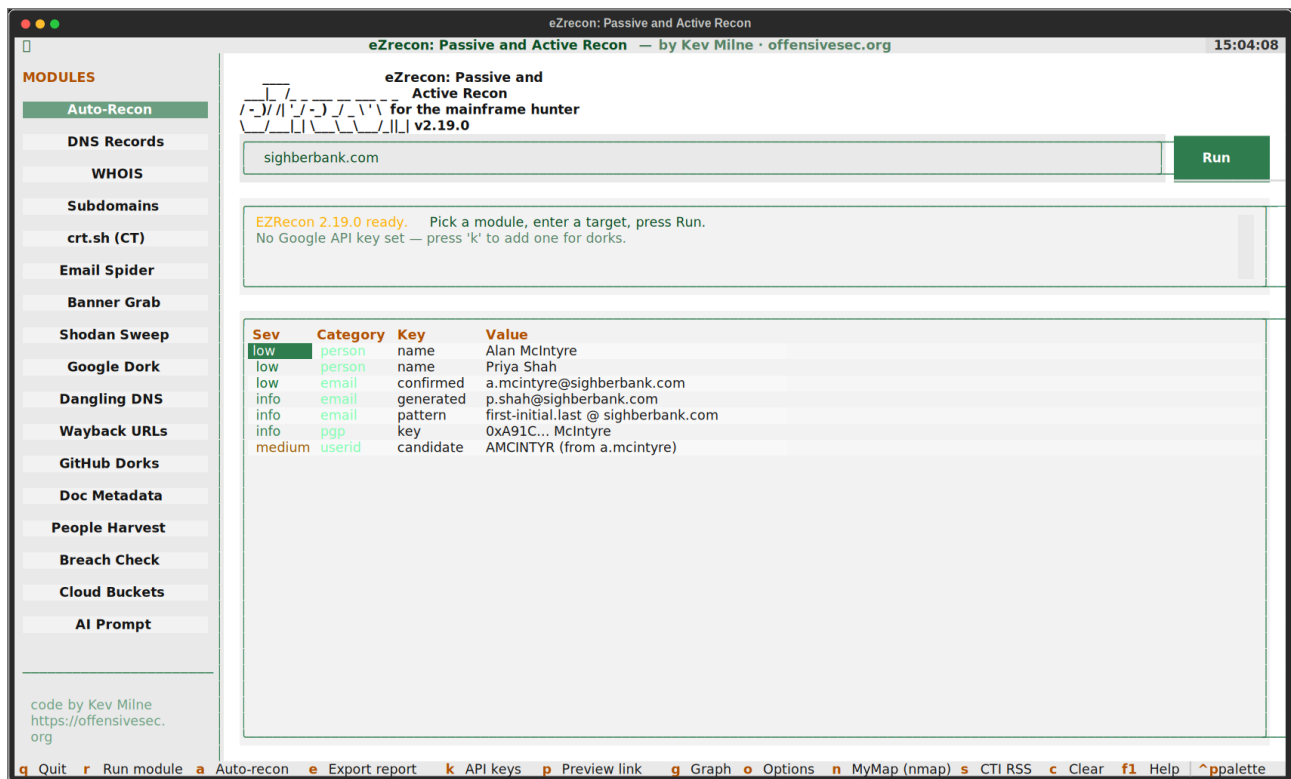


Figure 13. A people-harvest session: confirmed and generated addresses, the inferred naming pattern, a PGP key, and a candidate userid derived from an address.

Breach check

Checks discovered emails against Have I Been Pwned and reports which appear in known breaches and what was exposed.

Why we run it. An address that turns up in a breach that leaked passwords is a direct lead for a credential test against the target's logins. Even a data-only breach informs a phishing pretext. This is where OSINT crosses into an access path.

What you're looking for. Addresses that appear in breaches, and whether those breaches leaked passwords — a direct lead to a credential test.

Run it

```
ezrecon breach <domain> | --email <addr>
```

In the interface: Breach Check module (needs a key)

Needs an HIBP key in `hibp_api_key`. Requests are paced to respect the rate limit.

Exposure — the open doors

These look for the mistakes that expose data or hand over a hostname: forgotten DNS pointers and misnamed cloud buckets. One is passive by default; the other probes the cloud providers and is active.

Dangling DNS / takeover

Checks discovered subdomains for a CNAME pointing at a decommissioned cloud service we could re-register and take over. It classifies by provider fingerprint and, with `--http`, confirms by fetching.

The check is careful about false positives: it classifies by the fingerprint of the pointed-at service, and only with `--http` does it actually fetch to confirm. A confirmed takeover is high value because it lets an attacker serve content from a hostname the organisation still advertises.

Why we run it. A dangling record lets an attacker claim the hostname and serve content as the organisation, which is a foothold for phishing and cookie theft. Finding these is high value and the DNS-only check is completely passive.

What you're looking for. A subdomain whose CNAME points at a decommissioned service you could re-register and serve content from.

Run it

```
ezrecon takeover <domain> [--http] [--update]
```

In the interface: Dangling DNS module

The DNS classification is passive. The `--http` confirmation is active and gated. `--update` refreshes the provider fingerprints.

Cloud buckets

Generates candidate bucket names from the target's name plus environment suffixes (backups, db-dumps, terraform-state, prod, and so on) and checks them against S3, Google Cloud Storage and optionally Azure.

Bucket names are guessable because organisations are consistent, and the names advertise the contents. eZrecon builds candidates from the target name and a list of environment suffixes — prod, dev, backups, db-dumps, terraform-state — and checks them across the providers. Backups and database dumps are where sensitive mainframe extracts most often end up in the open.

Why we run it. Organisations name buckets the way they name everything else, which makes them guessable, and the names advertise the contents. A public bucket is an immediate data leak; one that merely exists is still attack surface. Backups and database dumps are a common way sensitive mainframe extracts end up in the open.

What you're looking for. A public bucket (an immediate leak) or a private one that merely exists (still attack surface) — backups and dumps especially.

Run it

```
ezrecon buckets <domain> [--org <org>] [--azure]
```

In the interface: Cloud Buckets module (active)

This probes the cloud providers, so it is active and gated. A public bucket is flagged high; a private but existing one is medium.

Support tools — the workbench

These do not find targets themselves. They help us reason about what we have found, keep an eye on the wider threat picture, and package a session for a second read.

AI OSINT prompt

Assembles a structured intelligence prompt from the current session, ready to paste into an assistant for a summary or next-step suggestions.

Why we run it. A long session produces a lot of findings. The prompt packages them so we can get a fast second read on what matters and what to do next.

What you're looking for. A fast, structured second read of a busy session and what to do next.

Run it

```
ezrecon prompt <domain>
```

In the interface: AI Prompt module

CTI RSS reader

A security-news reader over a set of feeds (Krebs, BleepingComputer, Planet Mainframe and more) that stars items mentioning the current target or mainframe keywords, and can email a selection.

Why we run it. Recon does not happen in a vacuum. Knowing that a technique or a vendor tied to our target is in the news this week shapes the engagement, and surfacing the relevant items automatically saves reading a dozen feeds by hand.

What you're looking for. News tied to the target, a vendor or a technique in the estate, surfaced automatically from the feeds.

Run it

```
ezrecon rss [--highlight <terms>] [--email]
```

In the interface: CTI RSS reader (press s)

Add your own feeds from the name and URL box in the reader, or with ``ezrecon rss --add-feed "Name" <url>``. They are saved to your config so they survive upgrades.

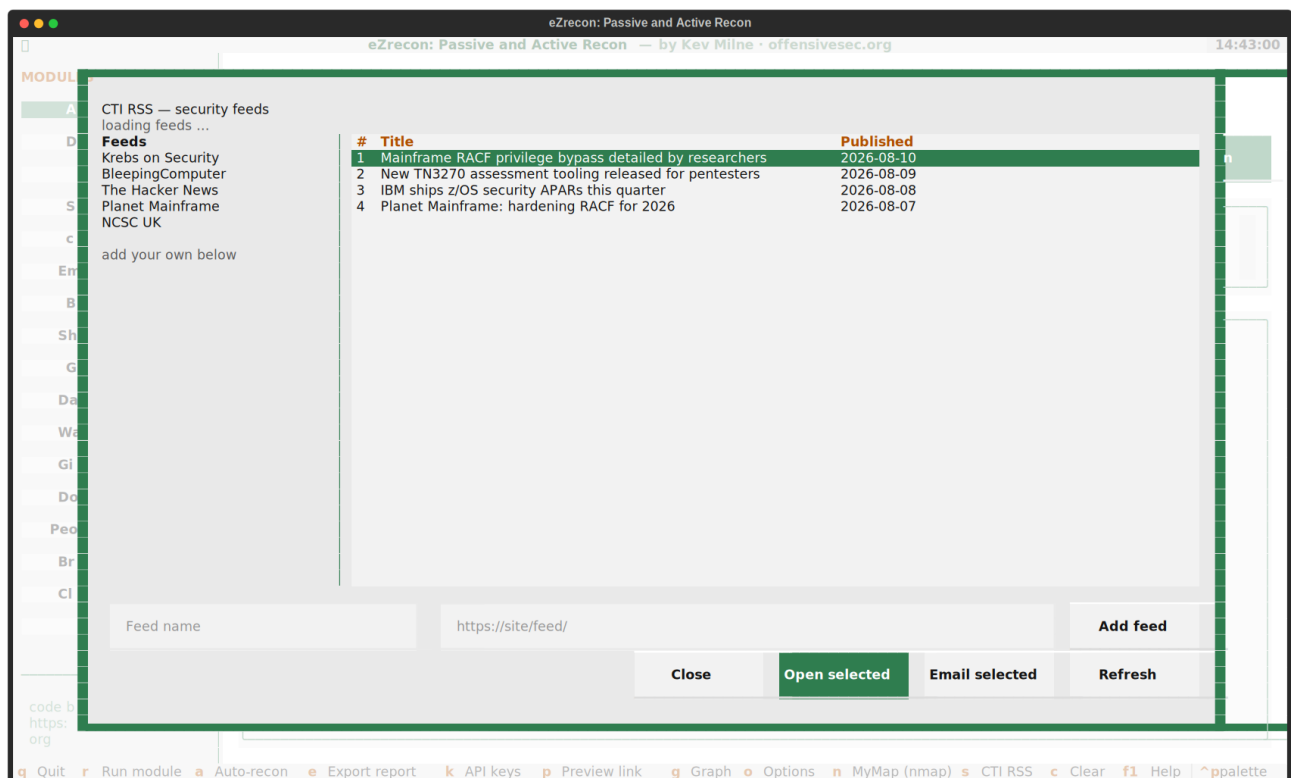


Figure 14. The CTI RSS reader, widened so the item titles are readable. Feeds are on the left, items on the right, and you can add your own feed or email a selection.

Part V · MyMap, the nmap script runner

MyMap is the active half of eZrecon: a menu-driven nmap script runner, opened with the `n` key. It is a port of the standalone MyMap, with the same idea of putting the whole NSE library at your fingertips without having to remember script names or argument syntax. Because running a scan touches the target, MyMap is gated: the first time you press Run it warns you and asks you to confirm, and only then does it scan.

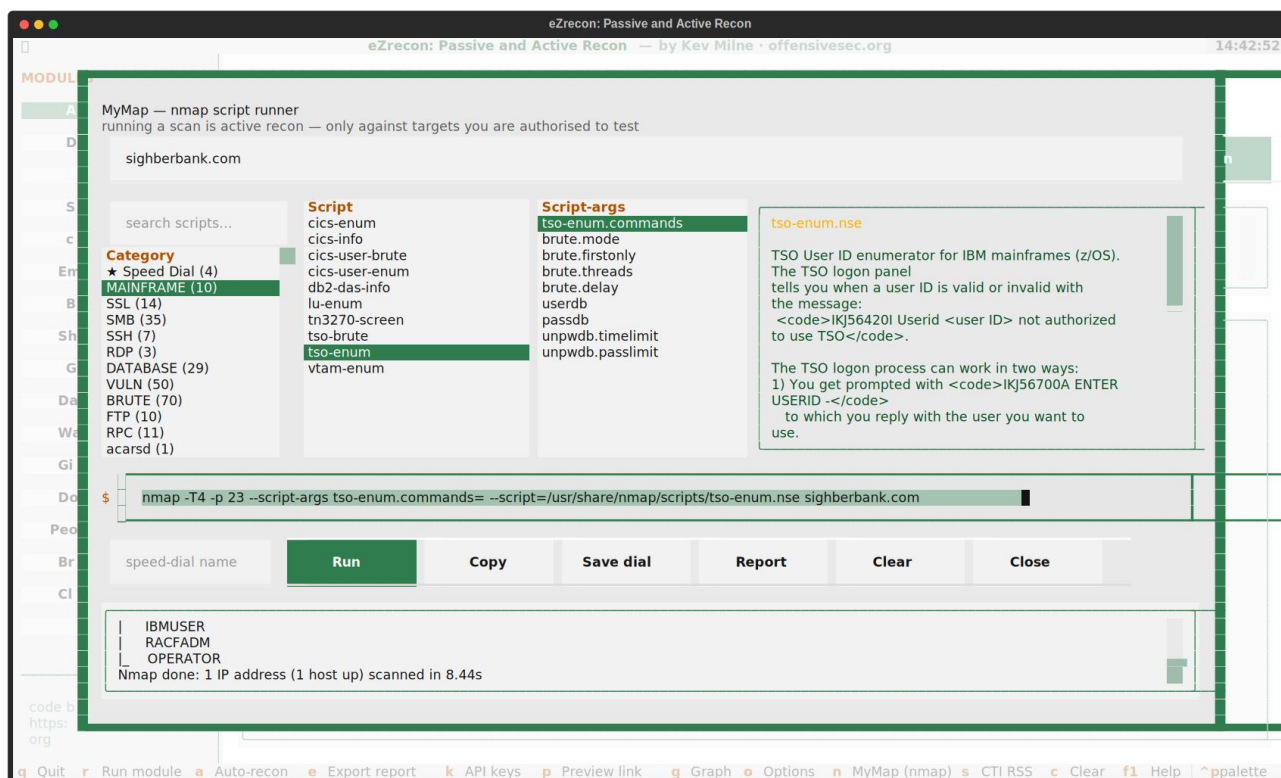


Figure 15. MyMap with the MAINFRAME category open and tso-enum selected. The description and the script's arguments are on the right, the editable command line runs across the middle, and the scan output, with the valid user IDs highlighted, fills the pane below.

The layout

The screen is three picker columns and a description, over a single editable command line. The first column lists the categories, including the sub-menus we care about most: MAINFRAME, SSL, SMB, SSH, RDP, DATABASE, VULN, BRUTE, FTP and RPC, plus your saved speed dials. The second column lists the scripts in the chosen category. The third column lists that script's `--script-args`, read from the NSE database: the arguments documented in the script and the ones it references in its code, along with the common library arguments (`userdb`, `passdb` and the brute settings) where they apply. The pane on the right shows the script's description, taken straight from the NSE database, so you can read what it does before you run it. A search box at the top of the first column filters the whole library by keyword.

Building and running a scan

The three columns are there to show you what you can pick; the thing you actually run is the editable command line in the middle. Pick a category and a script and the line fills in with a sensible default. Pick an argument from the third column and it is inserted into the line as `--script-args name=`, ready for you to type the value. Set the target and ports in the small boxes at the top, or simply type them onto the command line yourself; the line is the single source of truth, so anything you can pass to nmap you can

add there. When you are ready, press Run to confirm and scan. The output streams into the pane below with the interesting lines highlighted, and Report writes a short findings paragraph from what came back.

NOTE The command line is what runs, not the target box. Add extra switches and ports straight onto the line, or into the extra flags box. Do not put them in the target box, or they become part of the hostname and the scan fails to resolve.

Reading the output and the report

As a scan runs, its output streams into the pane along the bottom, and MyMap highlights the lines worth your eye: anything that looks vulnerable, out of support, weak or deprecated is coloured so it stands out of the noise. When the scan finishes, press Report and MyMap reads back over the output and writes a short paragraph — whether it saw anything of concern, and the lines that prompted it. It is not a substitute for reading the output yourself, but it is a fast first read and a tidy thing to paste into notes.

Speed dials and copying

Speed dials save a command's flags under a name so you can reuse them. Type a name, press Save dial, and it is stored to your config; select the Speed Dial category to load one back. A few mainframe-flavoured dials ship by default. Copy sends the built command to the main log instead of running it, which is the passive path when you would rather run the scan yourself.

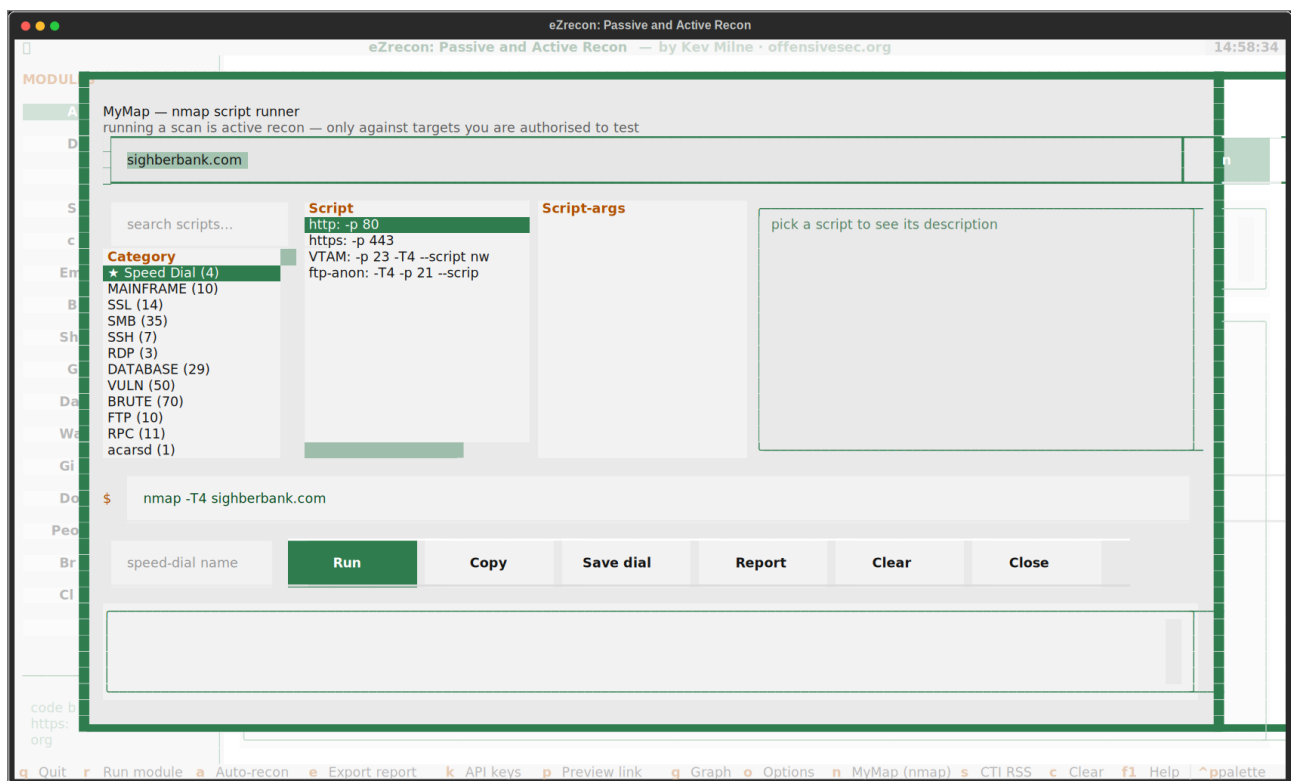


Figure 16. The Speed Dial category. Saved commands, seeded with a few mainframe-flavoured defaults, load back with a click for reuse.

A few dials worth keeping for mainframe work:

Dial	Flags and purpose
mainframe-terminal	-p 23,992 --script tn3270-screen — A quick look at the terminal screen on both TN3270 ports.

tso-ids	-p 23 --script tso-enum — Enumerate TSO user IDs against the terminal port.
cics-region	-p 23 --script cics-info — Read a CICS region through CEMT in one shot.
db2-das	-p 523 --script db2-das-info — Unauthenticated DB2 Administration Server profile.

Searching the library

With hundreds of scripts across the categories, the search box at the top of the first column is often the fastest way in. Type a few characters and the script list narrows to everything matching, across every category at once, so you can go straight to the one you want.

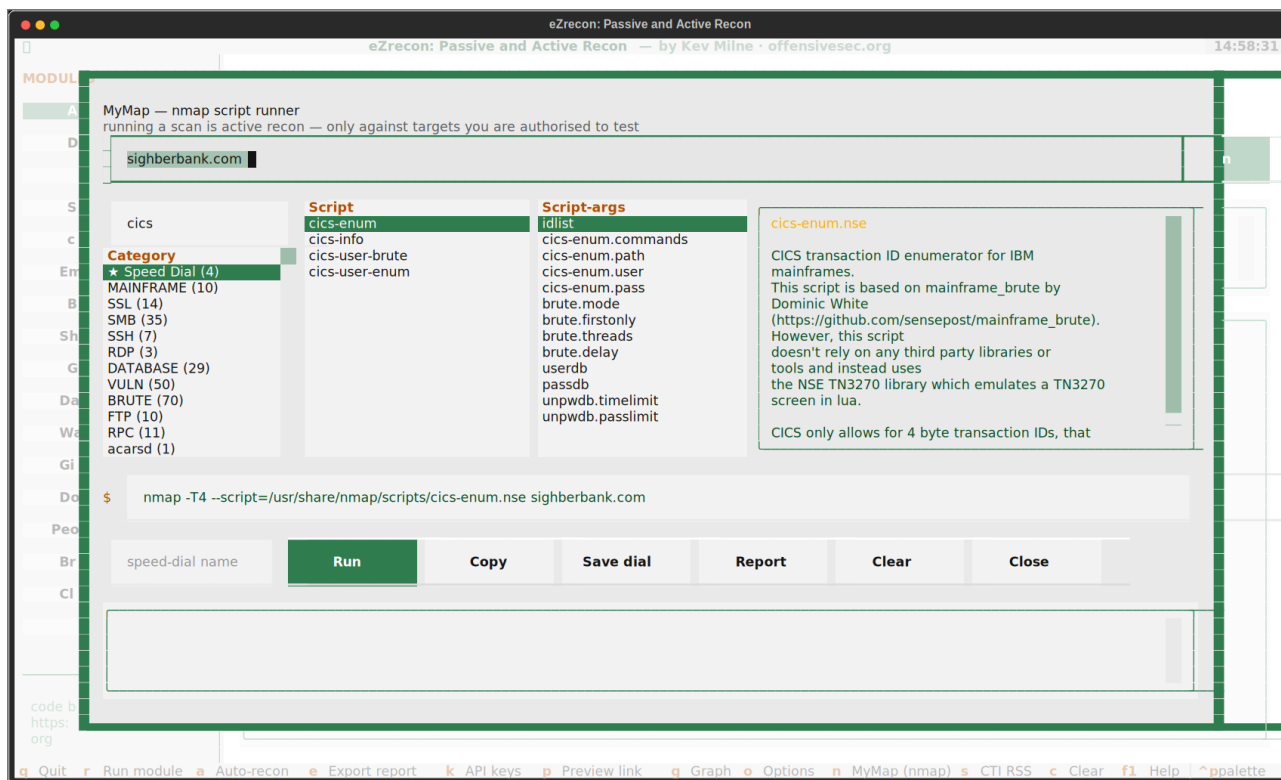


Figure 17. The MyMap search filtering the whole NSE library by keyword. Typing a few characters narrows hundreds of scripts to the handful you want.

A worked example

Open MyMap with n, pick the MAINFRAME category, and select tso-enum. Its description explains that the TSO logon panel tells us when a userid is valid, and the third column lists the script's arguments, tso-enum.commands among them. Pick that argument to drop it onto the command line, set the target, and pin the terminal port:

```
nmap -T4 -p 23 --script-args tso-enum.commands= --script=/usr/share/nmap/scripts/tso-enum.nse TARGET
```

Fill in the argument value on the line, press Run once to arm and once more to confirm, and the output streams in below with the interesting lines highlighted. Press Report for a short summary, or Save dial to keep the command for next time.

Part VI · The mainframe NSE scripts

This is the part of MyMap that earns its place on a mainframe engagement. nmap ships a small but potent set of scripts written for IBM mainframes, most of them driving a real TN3270 terminal emulated in Lua, and they are the difference between knowing a host answers on port 23 and knowing which user IDs, transactions and applications live behind it. We group them by what they touch: the terminal itself, TSO, CICS and DB2. Every one of them is active, so run them only against a system you are authorised to test.

NOTE The example output shown for these scripts is illustrative — the shape of a real run against a live system — so you know how to read what comes back.

Mainframe services and ports

It helps to know which ports carry which mainframe services, because that is what the banner grab and MyMap key off. These are the ones worth watching on a z/OS estate.

Port	Service
23 / 992	TN3270 / TN3270 over TLS — The terminal. Everything the mainframe scripts drive lives here — VTAM, TSO, CICS logon screens.
623	TN3270 (alternate) — A second port tso-enum and tso-brute will also try for the terminal service.
446 / 50000	DRDA / DB2 — The database protocol clients use to reach DB2. Confirming these is listening tells you a DB2 subsystem is live.
523	DB2 Administration Server — Answers db2-das-info unauthenticated with the server profile and version.
21	FTP — Often present on z/OS and frequently misconfigured; anonymous access and dataset browsing are classic findings.
175 / 1414	JES NJE / MQ — Network Job Entry and MQ, where they are exposed, are their own rich attack surface.

Things they share

The TN3270 library. Most of these scripts do not speak a simple protocol; they emulate a 3270 terminal, read the screen, type into fields and read the result, exactly as an operator would. That is why they are slower than an ordinary script and why they can reach things a port scan never will.

userdb and passdb. The enumeration and brute scripts take a userdb and a passdb: your wordlists of candidate user IDs and passwords. This is where the people harvest pays off. The naming convention it inferred and the author usernames from document metadata become the userdb you feed here.

the .commands argument. On a real system you rarely land straight on the screen you want. You come up on a VTAM menu or a session manager and have to navigate to TSO or CICS first. The per-script .commands argument is a semicolon-separated list of keystrokes the script sends to get there before it starts work. Getting these right for the target is often the whole game.

brute settings. The brute scripts share the standard settings: brute.mode, brute.threads and brute.delay. Go gently. Mainframe logons can lock accounts and the session managers are not built for a dozen parallel screens, so a low thread count and a delay are usually the right call.

The order to run them

These scripts build on each other. A sensible order saves time and keeps you gentle:

- Start with `tn3270-screen` to see what the terminal presents and where you land. Everything after depends on knowing that.
- If you land on a session manager or a fixed application, try `lu-enum` to find a Logical Unit that takes you somewhere more useful.
- Map the applications behind the VTAM screen with `vtam-enum`, so you know which regions (CICS, IMS, TSO) exist.
- Enumerate user IDs where you can — `tso-enum` for TSO, `cics-user-enum` for CICS — to build a `userdb` from real IDs rather than guesses.
- Read a CICS region with `cics-info` if CEMT is reachable; it hands you datasets, transactions and user IDs in one go.
- Only then brute — `tso-brute` or `cics-user-brute` — with the confirmed IDs and a considered `passdb`, gently, mindful of lockout.
- Alongside all of this, `db2-das-info` on 523 confirms a DB2 subsystem unauthenticated, before you go near DRDA.

Reading the terminal

Before anything else we want to see what the terminal shows and what sits behind it. These three work at the TN3270 layer.

tn3270-screen · ports 23, 992

Connects to the TN3270 service and returns the screen it is presented with, listing any hidden fields underneath with their row and column.

What you're looking for. This is the first thing to run against a new mainframe host. The screen tells you what you are looking at (a VTAM menu, a session manager, a logon panel) and the hidden fields often leak application names and prompts that shape everything after. No credentials, no noise, just a look.

Key arguments

`tn3270-screen.commands` — keystrokes to send before capturing the screen

`tn3270-screen.lu` — request a specific Logical Unit

`tn3270-screen.disable_tn3270e` — fall back to basic TN3270

Example

```
nmap -p 23 --script tn3270-screen --script-args tn3270-screen.commands=VALUE TARGET
```

A run looks like

```
| tn3270-screen: | *** Welcome to SIGHBERBANK VTAM *** | Application ==> | (hidden
field @ row 21, col 14) |_
```

Reading it. The screen text names the environment and the prompt; the hidden field location is where you would type an application ID.

lu-enum · ports 23, 992

Enumerates the Logical Units of a TN3270E server. When you connect you are handed an LU from a pool, or dropped at a fixed application; this script guesses valid LUs that bypass the default.

What you're looking for. The LU you are given may drop you at a locked-down session manager, while another LU takes you straight to TSO or CICS. Finding those LUs can walk you around the front door entirely. You are looking for LUs that land you somewhere more useful than the default.

Key arguments

`lulist` – path to your list of LUs to try

`lu-enum.path` – folder to save screen captures of valid LUs

Example

```
nmap -p 23 --script lu-enum --script-args lulist=VALUE TARGET
```

A run looks like

```
| lu-enum: | Logical Units that bypass the default: | TCP00042 -> TSO | _ TCP00105
-> CICSTS55
```

Reading it. Each line is an LU that lands you somewhere other than the default session manager. TCP00042 takes you straight to TSO — a way around the front door.

vtam-enum · ports 23, 992

Brute-forces VTAM application IDs. Many mainframes put a VTAM screen in front of everything, and each application (CICS, IMS, TSO and the rest) sits behind an application ID of up to eight characters.

What you're looking for. You are looking for the application IDs that VTAM will connect you to. Every valid ID is a door: a CICS region, an IMS system, a TSO logon. Feed it a good idlist and it maps the applications the VTAM screen can reach.

Key arguments

`idlist` – path to your list of application IDs

`vtam-enum.macros` – treat entries as macros rather than raw IDs

`vtam-enum.commands` – keystrokes to reach the VTAM screen first

`vtam-enum.path` – folder to save captures of valid IDs

Example

```
nmap -p 23 --script vtam-enum --script-args idlist=VALUE TARGET
```

A run looks like

```
| vtam-enum: | Valid Application IDs: | CICSTS55 | IMSPROD | _ TSO
```

Reading it. Each line is a live application behind the VTAM screen — here a CICS region, an IMS system and TSO. Every one is a door to try next.

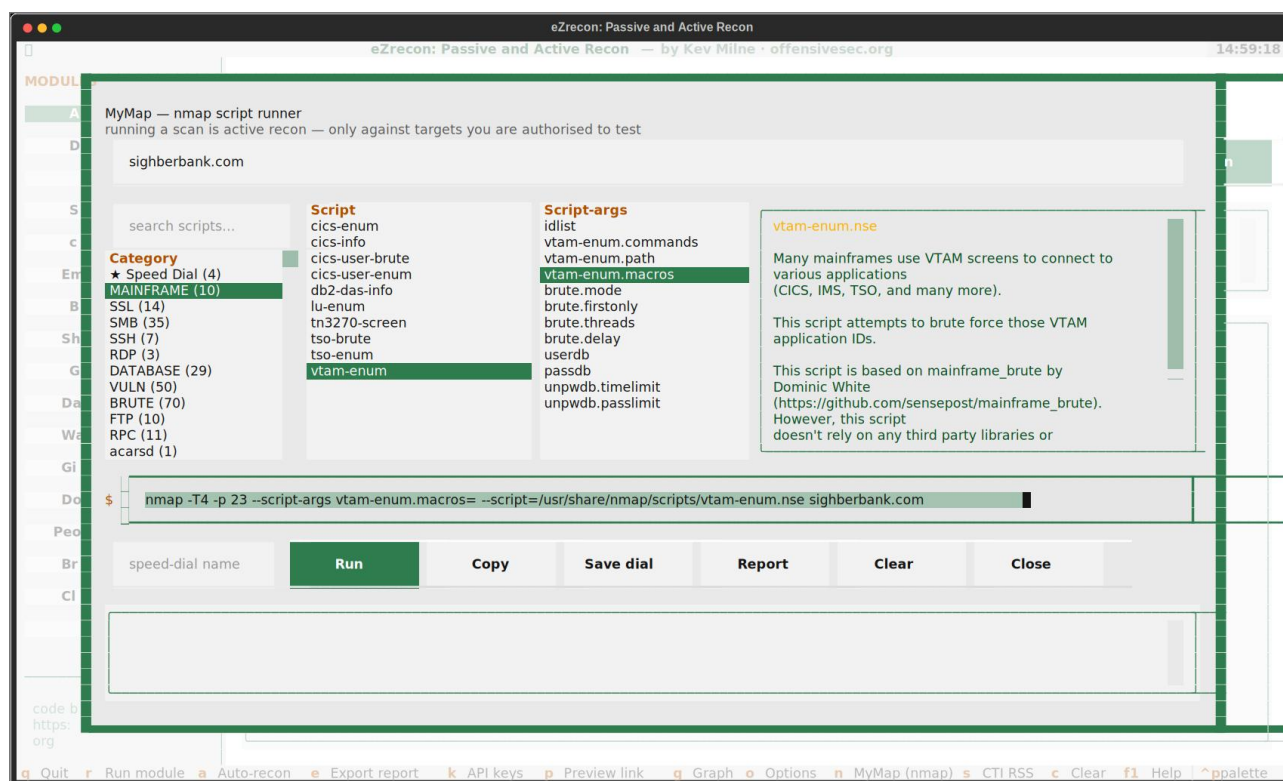


Figure 18. MyMap on vtam-enum, the VTAM application-ID brute forcer. The macros argument is picked onto the command line; the description notes it hunts the application IDs behind the VTAM screen (CICS, IMS, TSO and the rest).

NOTE Read the captured screens carefully. The text on a VTAM or session-manager screen names the applications and the keystrokes to reach them, and the hidden fields `lu-enum` and `tn3270-screen` surface often reveal prompts and application names you will need as the commands argument for the scripts that follow.

TSO — user IDs and passwords

TSO is where interactive users log on to z/OS. A valid TSO user ID is the first half of an account, and these two scripts find the IDs and then the passwords.

tso-enum · ports 23, 992, 623

Enumerates valid TSO user IDs. The logon path tells you when an ID is invalid with the message IKJ56420I Userid not authorized to use TSO, so the script can sort the real IDs from the rest without ever logging in.

What you're looking for. You are looking for the list of user IDs that exist on the system. This is pure enumeration, not a brute force: no passwords are tried, but the output is the `userdb` for `tso-brute` and a strong signal of the naming convention in use. Remember TSO IDs cannot start with a number, are alphanumeric plus `@` `#` `$`, and are at most seven characters.

Key arguments

`tso-enum.commands` — keystrokes to reach the TSO logon first

`userdb` — the list of candidate user IDs to test

Example

```
nmap -p 23 --script tso-enum --script-args tso-enum.commands=VALUE TARGET
```

A run looks like

```
| tso-enum: | Valid TSO User IDs: | IBMUSER | RACFADM | STCUSER
```


Reading it. These are confirmed user IDs — the ones the logon accepted as real. Save them as your userdb for tso-brute; no password was tried to get them.

tso-brute · ports 23, 992, 623

Brute-forces TSO accounts, driving the real logon screen through the TN3270 library. It respects the TSO ID rules and can be told how to behave when it lands a valid logon.

What you're looking for. You are looking for a working user ID and password. This is loud and it can lock accounts, so pair it with the IDs tso-enum confirmed and a considered passdb rather than a spray. always_logon controls whether the script completes the logon when it guesses a pair, which can kick a real user off, so use it deliberately.

Key arguments

userdb — candidate user IDs (ideally from tso-enum)

passdb — candidate passwords

tso-brute.always_logon — complete the logon on a hit (may evict a user)

brute.delay — pause between attempts to be gentle

Example

```
nmap -p 23 --script tso-brute --script-args userdb=VALUE TARGET
```

A run looks like

```
| tso-brute: | Accounts: | IBMUSER:SYS1 - Valid credentials |_ Statistics: 240
guesses in 60 seconds
```

Reading it. A hit is a working account. Treat IBMUSER:SYS1 as a live credential, record how you found it, and mind that completing the logon can evict a real user.

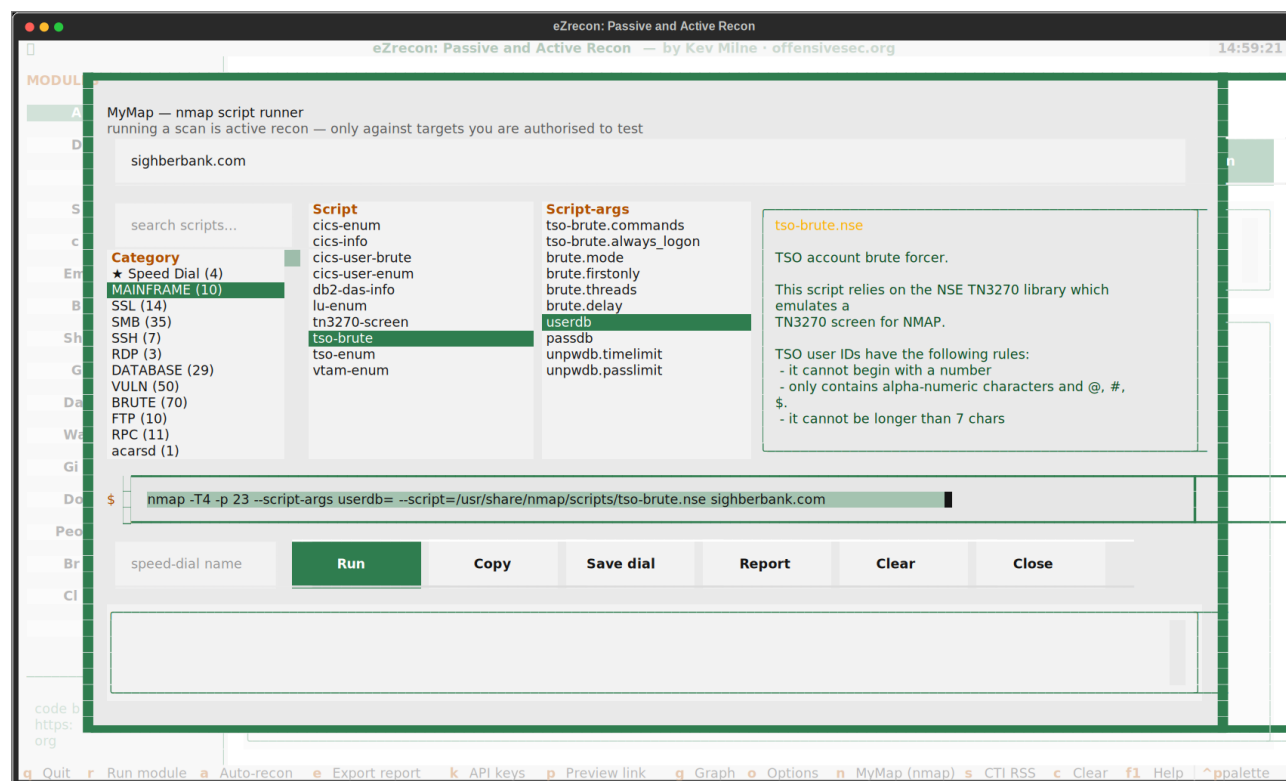


Figure 19. MyMap on tso-brute. Brute-force scripts expose the shared library arguments — userdb and passdb for the wordlists, and the brute settings for mode and threads — alongside the script's own options.

NOTE tso-enum's value is the list of confirmed IDs; save it as your userdb. A successful tso-brute hit is a working account — treat it with the same care you would any live credential, and record exactly when and how you used it.

CICS — the transaction monitor

CICS runs the transactions that are the business: the screens tellers and clerks use all day. These four read a region and test its logons.

cics-info · ports 23, 992

Uses the CEMT master terminal transaction to gather information about a CICS region: operating-system details, datasets, the defined transactions and the user IDs. It is based on Ayoub ELAASSAL's CICSspwn.

What you're looking for. This is the richest single read of a CICS region. You are looking for the datasets it can see (dataset names are gold on z/OS), the transactions defined, and the user IDs. If CEMT is reachable without authentication, this alone can map the region.

Key arguments

`cics-info.cemt` — the CEMT transaction ID (default CEMT)

`cics-info.commands` — keystrokes to reach CICS first (default cics)

`cics-info.user` — user ID if CEMT needs authentication

`cics-info.pass` — password if CEMT needs authentication

Example

```
nmap -p 23 --script cics-info --script-args cics-info.cemt=VALUE TARGET
```

A run looks like

```
| cics-info: |   Applid: CICSTS55   Security: ON |   OS: z/OS 02.05.00 |   Datasets:
CICS.SDFHLOAD, SYS1.PARMLIB |   Transactions: CEMT, CESN, CESL, PAY1 |_   Userids: IBMUSER,
CICSUSER
```

Reading it. One read of the region: the applid and OS level, the datasets it can see (names that map the data), the transactions defined, and the user IDs. PAY1 among the standard CEMT/CESN transactions is the kind of application transaction worth a closer look.

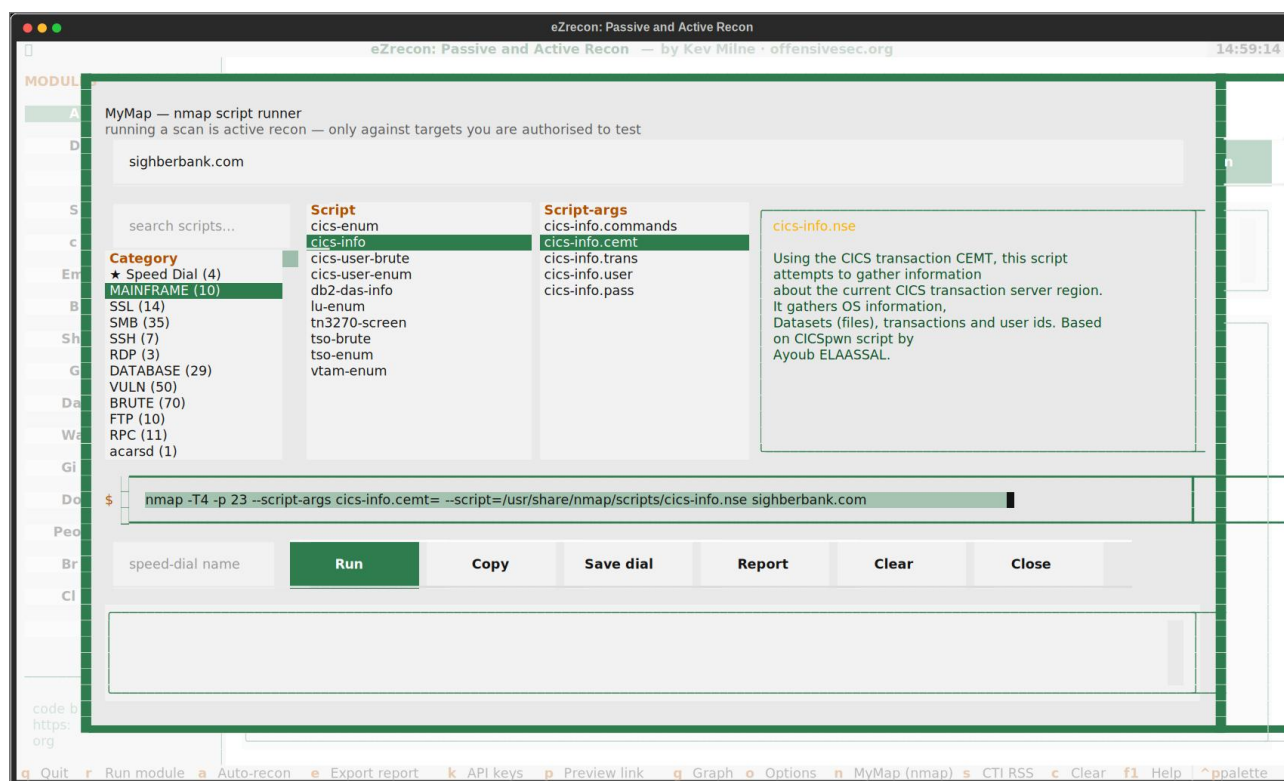


Figure 20. MyMap on cics-info. The third column lists the CICS arguments (the CEMT transaction, the access command, and optional credentials) and the description explains that the script uses CEMT to read the region's datasets, transactions and user IDs.

cics-enum · ports 23, 992

Enumerates CICS transaction IDs. CICS transaction IDs are exactly four characters, and this script works through a list to find which ones the region will run.

What you're looking for. You are looking for the transactions that exist and respond. Each is a screen or a function; some are administrative, some are the application. With authenticated access it can go further, hence the user and pass options.

Key arguments

- idlist – path to your list of transaction IDs
- cics-enum.user – user ID for authenticated enumeration
- cics-enum.pass – password for authenticated enumeration
- cics-enum.path – folder to save captures of valid transactions

Example

```
nmap -p 23 --script cics-enum --script-args idlist=VALUE TARGET
```

A run looks like

```
| cics-enum: | Valid Transaction IDs: | CEMT | CESN | _ PAY1
```

Reading it. The four-character transaction IDs the region will run. The standard ones are administrative; anything bespoke, like PAY1, is the application.

cics-user-enum · ports 23, 992

Enumerates CICS user IDs against the CESL or CESN login screen, the same way tso-enum does for TSO.

What you're looking for. You are looking for the valid user IDs the CICS logon will accept, which become the userdb for cics-user-brute and another read on the naming convention.

Key arguments

`idlist` – path to your list of user IDs

`cics-user-enum.transaction` – login transaction to use (default CESL)

`userdb` – candidate user IDs

Example

```
nmap -p 23 --script cics-user-enum --script-args idlist=VALUE TARGET
```

A run looks like

```
| cics-user-enum: | Valid CICS User IDs: | IBMUSER | CICSUSER |_ TELLER01
```

Reading it. Confirmed CICS user IDs from the CESL screen. As with TSO, these become the `userdb` for the brute step and another read on the naming convention.

cics-user-brute · ports 23, 992

Brute-forces CICS user IDs and passwords against the CESL login screen.

What you're looking for. You are looking for a working CICS logon. As with TSO, pair confirmed IDs with a considered `passdb`, keep the thread count low, and mind account lockout.

Key arguments

`userdb` – candidate user IDs (ideally from `cics-user-enum`)

`passdb` – candidate passwords

`brute.threads` – keep this low on a live region

Example

```
nmap -p 23 --script cics-user-brute --script-args userdb=VALUE TARGET
```

A run looks like

```
| cics-user-brute: | Accounts: | TELLER01:PASSWORD1 - Valid credentials |_
Statistics: 180 guesses in 45 seconds
```

Reading it. A working CICS logon. Pair confirmed IDs with a considered `passdb`, keep the threads low, and mind lockout on a live region.

NOTE `cics-info` is the richest single read: the datasets it lists are a map of the data, and the transactions tell you what the region does. Confirmed user IDs from `cics-user-enum` feed `cics-user-brute`, exactly as with TSO.

DB2 — the database

One script here, and it needs no terminal and no credentials at all.

db2-das-info · ports 523

Connects to the DB2 Administration Server on port 523 and exports the server profile. No authentication is required for the request, and with a version scan it will set the product and version.

What you're looking for. You are looking for the DB2 server profile and version, unauthenticated. It confirms a DB2 subsystem is present and tells you what you are dealing with before you go near DRDA on 446 or 50000.

Example

```
nmap -p 523 --script db2-das-info TARGET
```

A run looks like

```
| db2-das-info: | DB2 Administration Server: DB2ADMIN | Version: 12.1.5 |_ Platform:
z/OS
```

Reading it. The DB2 subsystem confirmed and versioned without a credential, which tells you what you are dealing with before any authenticated DB2 work.

NOTE A profile from db2-das-info confirms the subsystem and its version without a credential, which tells you what you are dealing with before any authenticated DB2 work.

Quick reference

Script	Ports and purpose
tn3270-screen	23, 992 — Capture the terminal screen and hidden fields
lu-enum	23, 992 — Find Logical Units that bypass the default
vtam-enum	23, 992 — Brute VTAM application IDs (CICS, IMS, TSO...)
tso-enum	23, 992, 623 — Enumerate valid TSO user IDs
tso-brute	23, 992, 623 — Brute TSO user IDs and passwords
cics-info	23, 992 — Read a CICS region via CEMT (datasets, trans, IDs)
cics-enum	23, 992 — Enumerate CICS transaction IDs (4 chars)
cics-user-enum	23, 992 — Enumerate CICS user IDs (CESL/CESN)
cics-user-brute	23, 992 — Brute CICS user IDs and passwords
db2-das-info	523 — Export the DB2 DAS server profile (no auth)

Part VII · Methodology and playbook

A recon flow from a bare domain

We start wide and passive. Run the Auto sweep first: it maps DNS, WHOIS, certificate transparency and brute-forced subdomains, enriches the IPs with their ASN, then runs the passive OSINT chain over what it found. In one command we go from a domain to a session that already holds the estate, the archived URLs, the document authors and a consolidated people dossier.

Then we read the graph, not the table. We look for the high-value nodes: a public bucket, a dangling record, a breached account, an exposed version-control directory. Those are the leads, and we pivot from them. A document author becomes a userid; feed it to the harvest to generate the rest of the addresses. An email becomes a breach lookup. A subdomain that looks like a gateway becomes a MyMap scan, once we have switched active recon on and confirmed our scope.

Only then do we reach for the active and keyed tools, deliberately and one at a time: bucket discovery, takeover confirmation, the Shodan sweep, the HIBP checks, the MyMap scans. By this point we are not guessing. We are confirming specific things about specific assets we already found.

How the Auto pipeline chains

The Auto sweep is ordered by data flow. The independent discovery stages run together because none depends on another. Then the dependent stages run in sequence: ASN enrichment needs the IPs discovery found; the takeover check needs the subdomains; the Wayback harvest feeds the document-metadata stage, because the archive is where most documents come from; and the people harvest runs last, because it consolidates everything the earlier stages produced and infers the naming convention from the whole set. The sweep stays passive and keyless throughout. Add `--light` for the core map without the OSINT chain.

A mainframe-focused playbook

For a mainframe target the prize is usually a valid userid and a way to reach a terminal or a service, so we bias our recon towards that. In the archive and the dorks, the JCL, COBOL and copybook hits are worth more than another marketing PDF, because they carry dataset names, job names and sometimes credentials. In the document metadata, the author usernames are candidate TSO and RACF userids. In the harvest, the naming convention turns a few names into a userid list. In MyMap, the MAINFRAME scripts and the TN3270 and DRDA ports are what we hunt for.

A worked engagement

The brief

We are handed a single domain, `sighberbank.com`, and an authorisation to assess its external footprint and the mainframe behind it. Nothing else. This is the shape of most engagements: a name, a scope, and a mandate to find what an attacker could.

Map the estate, passively

We type the domain and press a for Auto-Recon. In one pass it resolves the DNS (the IONOS mail exchangers and the ui-dns nameservers tell us who runs the mail and the DNS), reads the certificate logs and brute-forces subdomains, enriches the IPs with their ASN, then runs the passive OSINT chain. When it

settles we have the estate, the archived URLs, the document authors and a people dossier, and we have not touched the target once.

Read the graph, not the table

We press g. The graph shows the domain joined to its mail exchangers, a handful of subdomains, a document author, an archived JCL file, and — in red — a dangling record and a public backups bucket. The reds are the leads. The JCL file and the tso-looking subdomain are the mainframe threads. We pivot from the author node to a harvest.

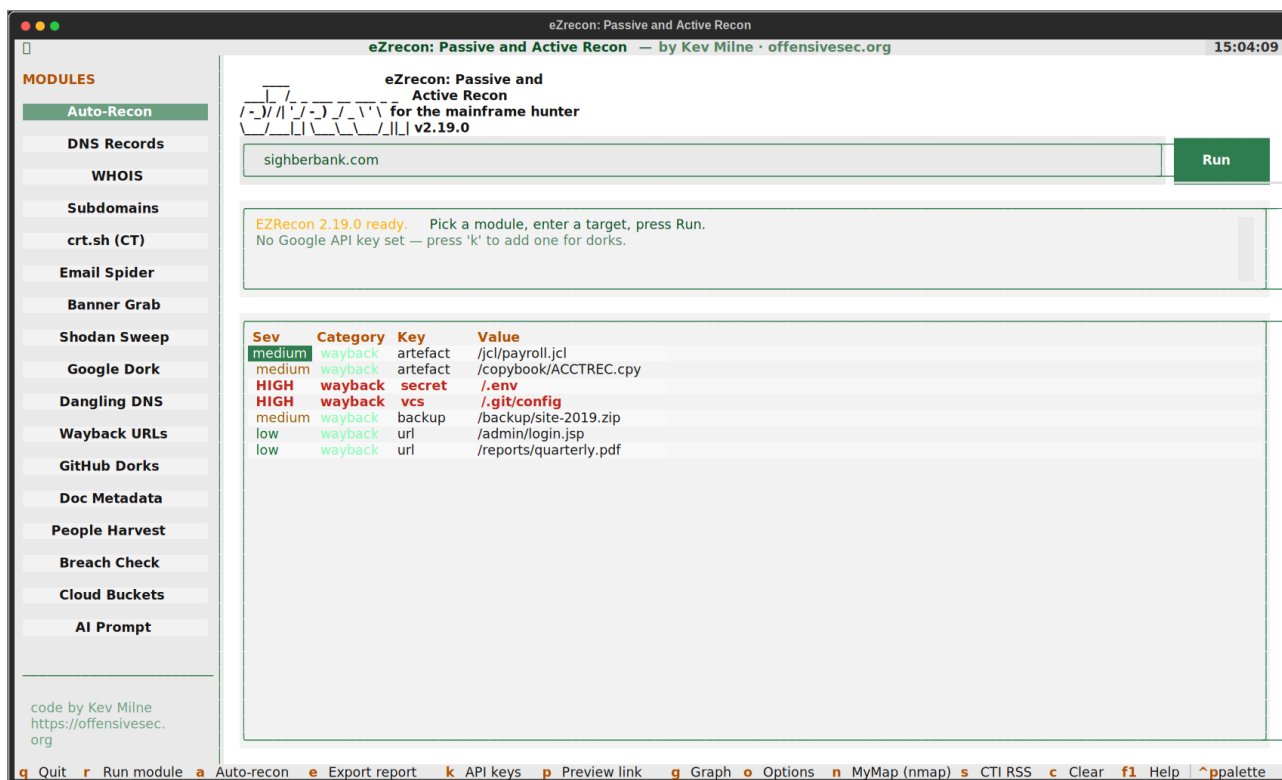


Figure 21. A Wayback harvest with the interesting paths flagged: JCL and copybook artefacts, an exposed .env and .git, a backup archive, and admin paths — several of them things active scanning could never see.

Turn names into user IDs

The harvest already inferred the naming pattern from the confirmed addresses, so the author's name becomes an address and the address becomes a candidate RACF user ID. A handful of names becomes a userdb. This is the list we will take to the mainframe.

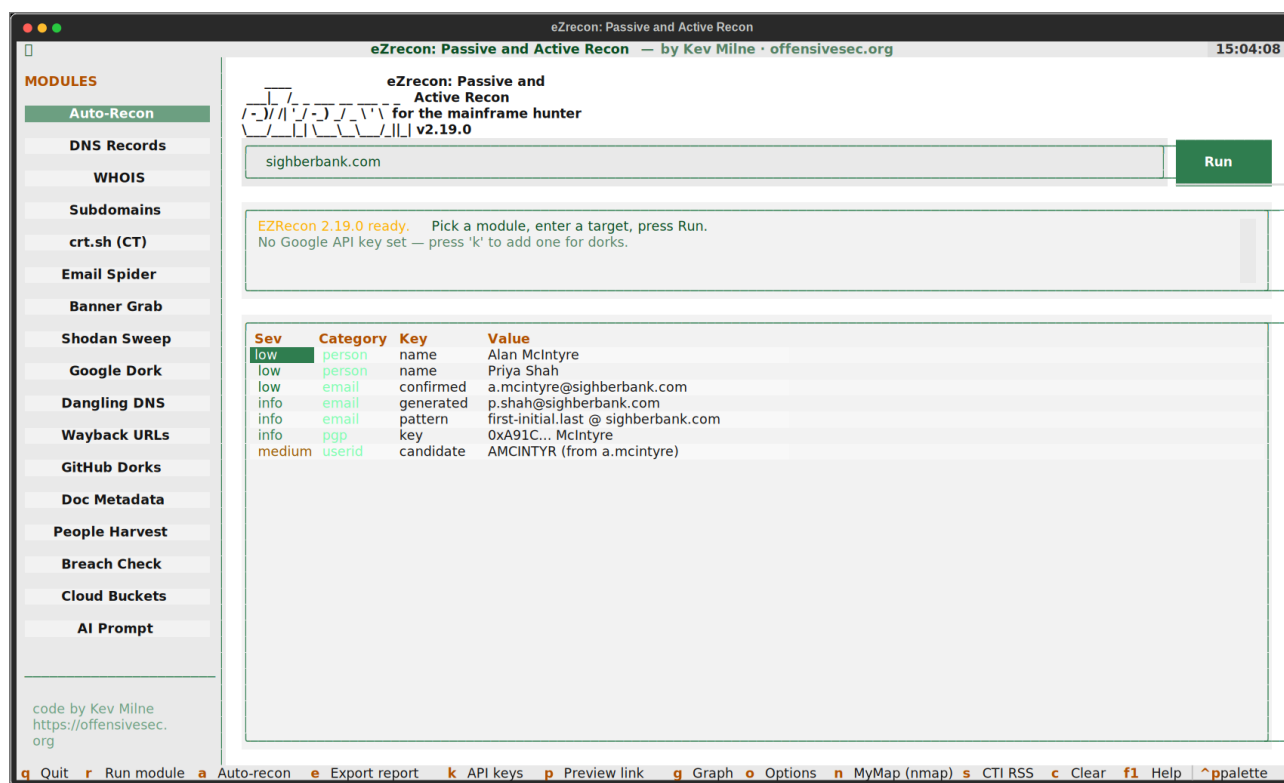


Figure 22. A people-harvest session: confirmed and generated addresses, the inferred naming pattern, a PGP key, and a candidate userid derived from an address.

Confirm the mainframe surface

Now we go active, deliberately, inside scope. We open MyMap (n) and run tn3270-screen against the gateway host to see what the terminal shows. It is a VTAM screen, so we note the keystrokes to reach TSO and feed them to tso-enum as its commands argument, with our harvested userdb. The output highlights the valid IDs. We have gone from a domain to a list of real mainframe user IDs, every step inside authorisation.

Write it up

We press e to export the session — the findings, the graph and the queries — and we have the evidence trail behind every claim. The tool found the doors and recorded them; the report is the story of which ones were open and what that means.

Part VIII • Reference

Command-line reference

Every capability is a subcommand. The interface and the shell drive the same engine, so anything here can be scripted.

Command	What it does
<code>auto <domain></code>	Run the full passive sweep and OSINT chain
<code>dns <domain></code>	DNS records and zone-transfer attempt
<code>whois <domain></code>	Registration record
<code>subdomains <domain></code>	Wordlist subdomain brute force
<code>crtsh <domain></code>	Certificate-transparency hostnames
<code>asn <host></code>	ASN and netblock lookup
<code>ports <host></code>	Banner grab on mainframe ports (active)
<code>shodan <domain></code>	Shodan sweep (needs key)
<code>favicon <host></code>	Favicon hash and Shodan pivot
<code>dork <domain></code>	Build and run mainframe dorks
<code>searx setup status stop</code>	Manage the local SearXNG
<code>github <domain> --org <org></code>	GitHub code-search dorks
<code>wayback <domain></code>	Archived URLs with artefact flags
<code>docmeta <domain> --url</code>	Document metadata extraction
<code>email <domain></code>	Email spider
<code>harvest <domain> --deep</code>	Consolidated people dossier
<code>breach <domain> --email</code>	HIBP breach check (needs key)
<code>takeover <domain> [--http]</code>	Dangling-DNS / takeover check
<code>buckets <domain> --org <org></code>	Cloud bucket discovery (active)
<code>prompt <domain></code>	Build an AI OSINT prompt
<code>rss [--add-feed ...]</code>	CTI RSS reader and feed management
<code>config set set-setting show</code>	Manage keys and settings

Severity levels

High — A concrete, high-value exposure or a direct lead to access: a public bucket, a confirmed takeover, a breached account with a leaked password, an exposed secret or version-control directory, a mainframe source artefact. High findings are drawn in red so they are impossible to miss.

Medium — Attack surface that is not yet an open door: a private bucket that exists, a takeover candidate, a breach without credentials, document authors.

Low — Useful context: a resolved subdomain, a discovered email, a data endpoint.

Info — Inventory and summaries: record counts, notes, and things worth keeping but not acting on directly.

Interface key bindings

Key	Action
r	Run the selected module against the target
a	Run Auto-Recon
g	Open the entity graph
n	Open MyMap, the nmap script runner
s	Open the CTI RSS reader
p	Preview a dork result
o	Options and appearance (themes)
k	The API-key vault
e	Export a report
F1	Contextual help and labs
c	Clear the results table
q	Quit

The interface at a glance

Every screen and how to reach it:

Screen	What it is
Main window	the modules, target box, status log and findings table
Auto-Recon (a)	the full passive sweep and OSINT chain
MyMap (n)	the nmap script runner
Entity graph (g)	the session as a navigable graph, with pivots
Google Dork	the dork selection and runner
Shodan	the Shodan query builder and runner
Preview (p)	a quick raw search of a dork result
CTI RSS (s)	the security-news reader and feed manager
API keys (k)	the secret-key vault
Options (o)	themes and run-time settings
Help / Labs (F1)	contextual help and hands-on labs

Glossary

A quick reference to the mainframe and recon terms this manual uses. If you come from the distributed world, start here.

z/OS — IBM's flagship mainframe operating system, the environment behind most of the services eZrecon and MyMap look for.

LPAR — Logical Partition. A mainframe is carved into LPARs, each running its own operating system, so one box can host many isolated systems.

TN3270 — The terminal protocol used to reach mainframe screens, a descendant of the 3270 family. MyMap's mainframe scripts drive an emulated 3270 terminal over it, usually on port 23 or 992.

LU — Logical Unit. The endpoint a TN3270E server assigns you; different LUs can drop you at different applications, which is why lu-enum matters.

VTAM — IBM's networking layer that fronts mainframe applications. A VTAM screen or session manager is often the first thing you meet, and each application sits behind an application ID.

TSO — Time Sharing Option. The interactive environment where users log on to z/OS with a user ID and password. A valid TSO ID is the first half of an account.

CICS — Customer Information Control System. The transaction monitor that runs the interactive business applications — the screens staff use all day. CESL and CESN are its logon transactions; CEMT is the master terminal.

IMS — Information Management System. Another transaction and database environment that often sits behind a VTAM application ID.

RACF — Resource Access Control Facility. The security manager on z/OS that holds the user IDs, passwords and access rules. The user IDs you enumerate are RACF user IDs.

JCL — Job Control Language. The scripting that submits batch work on z/OS. A JCL file found in the archive or a repository can carry dataset names, job names and sometimes credentials.

Copybook — A shared COBOL data-structure definition (a .cpy file). Finding one reveals record layouts and field names from the applications.

Dataset — The mainframe's term for a file. Dataset names are hierarchical and meaningful, so a list of them maps the data on a system.

DB2 / DRDA — IBM's mainframe database and the protocol clients use to reach it. DB2 listens on 446 and 50000; the DB2 Administration Server on 523 answers db2-das-info without authentication.

NSE — Nmap Scripting Engine. The framework the MyMap scripts are written in, including the Lua TN3270 library that makes the mainframe scripts possible.

Certificate transparency — Public logs of every issued TLS certificate. Reading them (crt.sh) is the richest passive source of hostnames.

Subdomain takeover — Claiming a hostname whose DNS still points at a service that has been deleted, letting you serve content as the organisation.

Troubleshooting

The interface looks broken or wraps oddly

The terminal is too narrow. Widen the window; eZrecon wants 120 columns or more, and MyMap a little more still. Over SSH, make sure your client is passing the real terminal size.

A dork or preview run comes back empty

The search engines have rate-limited themselves on a burst. eZrecon paces the queries and retries, but you can widen the pace with config set-setting searx_pace 3, or stand up your own SearXNG for a private backend.

MyMap says nmap not found

Install nmap and reopen MyMap. It reads the NSE scripts straight from /usr/share/nmap/scripts, so the scripts appear as soon as nmap is present.

A MyMap scan fails to resolve the target

A switch has ended up in the target box and become part of the hostname. Put switches and ports on the command line or in the extra flags box, never in the target box — the command line is what actually runs.

A mainframe script returns nothing useful

You probably need to navigate to the right screen first. Run `tn3270-screen` to see where you land, then set the script's `commands` argument to the keystrokes that reach TSO or CICS before the script starts.

A brute forcer is locking accounts

Slow down. Lower `brute.threads`, add a `brute.delay`, and pair the brute with the IDs an enumeration script already confirmed rather than a broad spray.

A keyed feature says no key is set

Add the key on the keys screen (k) or with `config set`. Shodan, HIBP, the Google API and the GitHub API each need their own; everything else works without one.

A closing word

eZrecon finds the doors. It maps the estate, gathers the people, and flags the exposures, quietly and mostly for free, and when we need to go active MyMap is there to do it in a controlled way. What it will not do is decide for us what is in scope or what is safe to act on. That judgement is the job, and it stays with us. Recon well, stay inside your authorisation, and let the findings lead.